



SurIoT: Automatic generation of Internet-of-Medical-Things Network Intrusion Detection and Prevention Rules

Lerina Aversano¹ · Stefano Galantucci¹ · Andrea Porcelli¹

Received: 3 March 2026 / Accepted: 5 June 2026
© The Author(s) 2026

Abstract

The Internet of Medical Things (IoMT) is transforming patient and healthcare management by enabling continuous monitoring, real-time decision support, and remote therapeutic interventions. However, the critical nature of connected medical devices, coupled with the use of lightweight protocols such as MQTT and CoAP, exposes these ecosystems to significant cybersecurity risks. Traditional Intrusion Detection/Prevention System (IDS/IPS) solutions, typically designed for enterprise networks, are often ineffective in IoMT contexts; conversely, complex Artificial Intelligence-based approaches, while accurate, demand computational resources incompatible with resource-constrained e-health environments.

This work introduces SurIoT, an explainable and lightweight approach for the automated generation of Suricata and Snort rules aimed at detecting threats in IoMT network traffic within simulated IoT/IoMT environments. The system extracts low-level network features directly from raw packet captures and utilizes a Decision Tree to derive interpretable detection logic. These decision boundaries are then translated into deployable Suricata and Snort rules, allowing for seamless integration into existing monitoring infrastructures while ensuring decision transparency.

Evaluation across four datasets, three targeted simulated attack scenarios (Crafted IoMT, DoS Flood, and Slow DoS) and one high-fidelity scenario with heterogeneous attacks (IoMT-Traffic-Data) demonstrates that the rules synthesized by SurIoT faithfully replicate the classification behaviour of the underlying Decision Tree, on every dataset. F1-scores reach 100% on Crafted traffic, 99.77% on DoS Flood, and 99.85% on the IoMT corpus; the Slow DoS scenario yields 83.08% F1, reflecting the inherent difficulty of separating slow-rate TCP attacks from legitimate handshake traffic at the per-packet level without temporal features. From an operational standpoint, Suricata processes the same traffic between $31\times$ and $57\times$ faster than the full machine learning inference pipeline, with no degradation in detection accuracy relative to the Decision Tree. SurIoT emerges as a technically feasible and explainable approach, providing a foundation for strengthening security in IoMT/IoT networks through automated rule synthesis.

Keywords Internet of Medical Things (IoMT) · Internet of Things (IoT) · Intrusion Detection System (IDS) · Intrusion Prevention System (IPS) · Suricata · Snort · Message Queuing Telemetry Transport (MQTT) · E-health · Attack Detection · Explainable Artificial Intelligence (XAI) · Decision Tree · Automated Rule Synthesis · Network Intrusion Detection and Prevention System (NIDPS) · Deep Packet Inspection

1 Introduction

The growing proliferation of the Internet of Medical Things (IoMT) [1–3] is profoundly revolutionizing patient and care management, enabling continuous patient monitoring, real-time clinical decision support, and remote diagnostics. Through IoMT networks, health data is continuously exchanged across a highly distributed architecture. While a central Hospital Information System (HIS) often accesses data from medical devices worn by patients to provide clinical support [4, 5], the ecosystem also involves numerous

✉ Stefano Galantucci
stefano.galantucci@unifg.it

Lerina Aversano
lerina.aversano@unifg.it

Andrea Porcelli
andrea.porcelli@unifg.it

¹ Department of Agricultural Sciences, Food, Natural Resources and Engineering (DAFNE), University of Foggia, Foggia 71121, Italy

distributed collection points, edge nodes, and diverse devices operating with heterogeneous protocols and firmware. This complex, decentralized data flow makes IoMT particularly relevant from a cybersecurity perspective. In emergencies or, more generally, where some form of medical intervention is useful or necessary, certain devices can act directly on the patient's health [6, 7]: for example, infusion pumps for diabetic patients can inject insulin, acting on hyperglycemia. Therefore Compromising the Confidentiality, Integrity, Availability (CIA) triad [8] could cause severe consequences: the effects of attacks can range from the theft of health data to the death of patients. European Union Agency for Cybersecurity (ENISA) Threat Landscape 2025 [9] reports that some attacks on e-health systems are of particular concern due to their impact.

IoMT devices have limited resources and, therefore, lighter communication protocols are employed. Among these, the most widespread and considered mature for use in Internet of Things (IoT) networks are Message Queuing Telemetry Transport (MQTT) [10, 11] and Constrained Application Protocol (CoAP) [12, 13]. This results in network traffic that is particularly different from that of corporate or private networks. Most of the literature on Intrusion Detection/Prevention Systems (IDS/IPS) [14–22] addresses these network types or the necessity to have specialized IDS, making such solutions weak for deployment in IoT and IoMT. Therefore, context-specific solutions must be developed. In recent years, much of the research in IoT and IoMT security has focused on the application of Artificial Intelligence (AI) and Deep Learning (DL) for anomaly detection and traffic classification [21–29]. While accurate, complex AI models often require high-end servers that are cost-prohibitive [30–32] for many healthcare facilities, and centralizing traffic analysis introduces significant network overhead and latency [32, 33]. Conversely, rule-based engines like Suricata [34] offer a lightweight, highly parallelizable, and efficient alternative perfectly suited for real-world IoMT constraints. In e-health environments, IDS/IPS solutions must comply [32, 33, 35] with latency constraints and guarantee speed, transparency, and smooth integration with existing network monitoring infrastructures. Literature [36, 37] highlights the need for fast, lightweight solutions capable of guaranteeing real-time monitoring and detection. In real-world cybersecurity architectures, signature-based and rule-based IDS/IPS systems are still cornerstones [38, 39] and are also considered relevant in the IoT context [40–46]. Suricata [34], in particular, is widely adopted due to its high-performance packet inspection capabilities, protocol awareness, and mature ecosystem. However, Suricata, like other Network-based Intrusion Detection and Prevention Systems (NIDPS), is currently unable to inspect CoAP traffic and offers limited support for MQTT traffic analysis (Section 2.5). Nevertheless, deploying and maintaining accurate

Suricata rules for MQTT is a non-trivial task, especially in the dynamic and heterogeneous context of IoMT traffic [41, 47, 48]. Manual rule definition is time-consuming, error-prone, and difficult to scale as device behaviors and attack patterns evolve.

To address these gaps, this paper proposes SurIoT, an approach capable of generating effective Suricata rules for attack detection in MQTT traffic. Specifically, the system employs packet dissection tools like Tshark [49] to parse raw network streams and extract granular features necessary for behavioral modeling [50, 51]. SurIoT leverages Decision Tree (DT) models to extract interpretable, transparent decision logic directly from observed traffic. This transparency, guaranteed by the DT, is particularly crucial in the IoMT landscape because the opacity typical of 'black-box' solutions presents an unacceptable risk; therefore, the literature emphasizes that security mechanisms must offer transparent decision boundaries to facilitate human verification [22, 52–54]. However, while literature offers several ML-based IDS for IoT [21, 22, 24–26], a fundamental scientific gap remains: the semantic mismatch between abstract ML predictions and deterministic rule enforcement in IoT/IoMT. Generative AI models attempt to bridge this gap through semantic translation of payloads [55, 56]; yet, they often lack the strict determinism required for critical medical infrastructure [33]. Furthermore, existing explainable approaches rely on stream-reassembled features, which introduce reassembly ambiguity and inherently force a generalized view of network traffic. [57] Overcoming these limitations, SurIoT architecturally constrains the extraction pipeline to ensure a lossless, 1:1 translation of its decision boundaries into deployable Suricata rules [34]. The generated rules are finally tested to verify that they maintain the same effectiveness as the initial model. The proposed approach offers a feasible alternative to purely AI-focused solutions, emphasizing operational efficiency and potential integration into existing security pipelines as demonstrated through high-fidelity simulations.

The main contributions of this work are:

- This work introduces SurIoT, an approach that lays the groundwork for adapting automated rule synthesis to the specific constraints of the MQTT protocol, offering an explainable approach evaluated within simulated IoMT/IoT ecosystems.
- SurIoT extends network monitoring capabilities by enabling stateless interception and analysis of MQTT traffic, addressing inherent vulnerabilities in IoMT/IoT messaging within a rule-based engine.
- Experimental results demonstrate that the rules synthesized by SurIoT effectively detect and mitigate anomalous behaviors within a high-fidelity simulated ICU scenario, ensuring robust protection for connected envi-

Table 1 List of acronyms used in this manuscript.

Acronym	Definition
ACK	Acknowledgment
AI	Artificial Intelligence
AIC	Akaike Information Criterion
AIDS	Anomaly-based Intrusion Detection System
ARP	Address Resolution Protocol
BERT	Bidirectional Encoder Representations from Transformers
BiGRU	Bidirectional Gated Recurrent Unit
BiLSTM	Bidirectional Long Short-Term Memory
CAM	Content Addressable Memory
CIA	Confidentiality, Integrity, Availability
CIDR	Classless Inter-Domain Routing
CoAP	Constrained Application Protocol
CSV	Comma-Separated Values
CV	Cross-Validation
CWR	Congestion Window Reduced
DDoS	Distributed Denial of Service
DL	Deep Learning
DNN	Deep Neural Networks
DNS	Domain Name System
DoS	Denial of Service
DT	Decision Tree
ECE	ECN-Echo
ECN	Explicit Congestion Notification
ENISA	European Union Agency for Cybersecurity
FHDT	Fuzzy Hoeffding Decision Trees
FIN	No more data from sender
FN	False Negative
FP	False Positive
FPR	False Positive Rate
GenAI	Generative Artificial Intelligence
GPU	Graphics Processing Unit
HIS	Hospital Information System
HTTP	Hypertext Transfer Protocol
ICU	Intensive Care Unit
IDS	Intrusion Detection System
IoMT	Internet of Medical Things
IoT	Internet of Things
IP	Internet Protocol
IPS	Intrusion Prevention System
JSON	JavaScript Object Notation
L2	Layer 2
L4	Layer 4
LLM	Large Language Models
MAC	Media Access Control
ML	Machine Learning
MQTT	Message Queuing Telemetry Transport
NB	Naive Bayes

Table 1 continued

Acronym	Definition
NIDPS	Network-based Intrusion Detection and Prevention Systems
NN	Neural Networks
OS	Operating System
PCAP	Packet Capture
PDU	Protocol Data Unit
pps	packets per second
PSH	Push function
QoS	Quality of Service
RAM	Random Access Memory
RF	Random Forest
ROC	Receiver Operating Characteristic
RST	Reset the connection
SVM	Support Vector Machine
SYN	Synchronize sequence numbers
TCP	Transmission Control Protocol
TN	True Negative
TP	True Positive
TPR	True Positive Rate
TTL	Time-to-Live
URG	Urgent pointer field
VRAM	Video Random Access Memory
XAI	Explainable Artificial Intelligence

ronments. The rules produced by SurIoT show a significant time and throughput advantage in recognizing (and blocking) attacks.

The remainder of this paper is organized as follows: Section 2 provides background information, Section 3 presents related works, Section 4 describes the proposed approach, Section 5 illustrates and discusses the experiments and their results, and finally, the conclusions are provided in Section 6.

Table 1 summarizes the list of abbreviations used in the manuscript.

2 Background

This section outlines the technical and theoretical foundations underpinning the proposed approach. First, the architectural principles of the MQTT protocol are examined, emphasizing its prevalence and specific constraints within IoMT environments. Subsequently, the operational mechanics of NIDPS are detailed, providing the rationale for adopting Suricata as the reference engine. Finally, the native capabilities and current limitations of Suricata regarding MQTT traffic inspection are analyzed to highlight the operational gaps addressed by this study.

2.1 MQTT protocol

MQTT [10] is a lightweight publish-subscribe protocol running over TCP/IP, specifically designed for constrained environments and low-bandwidth networks [10, 11, 58].

Unlike HTTP, MQTT uses a central *broker* to decouple publishers from subscribers. This allows for asynchronous communication where clients do not need to know each other's identities. Messages are simply routed to clients based on hierarchical *topics* (e.g., *home/sensor/temp*).

Reliability is handled via three Quality of Service (QoS) levels:

- **QoS 0:** Fire-and-forget; message loss is possible.
- **QoS 1:** Guaranteed delivery; duplicate messages may occur.
- **QoS 2:** Exactly-once delivery; highest overhead but no duplicates.

Because MQTT messages are often very small, the underlying operating system or network stack may buffer and transmit several messages in a single TCP segment or a tight burst. This process, known as *coalescing*, is defined as the broker's or client's ability to combine multiple small MQTT messages into a single TCP segment. Coalescing significantly alters the traffic shape. Intrusion Detection Systems

often rely on features like packet inter-arrival time and packet size to fingerprint devices or detect anomalies. Coalescing obscures these patterns: a periodic stream of small sensor readings may appear on the network as sporadic, larger bursts of data. This aggregation makes it difficult to distinguish individual message events and can mask the actual behavior of the IoT device [58].

2.2 NIDPS and Suricata

In cybersecurity, NIDPSs are employed to detect malicious traffic within a network. If such a system is also capable of blocking traffic, then it falls within the Intrusion Prevention System (IPS) family; otherwise, if it does not offer this functionality, it is referred to as an Intrusion Detection System (IDS).

The pipeline of an NIDPS [59] solution can be described as a sequence of modules: the process begins with packet acquisition from the network via the packet capturing module, which can operate in passive or inline mode, directly affecting performance based on real-time processing requirements. The captured packets are then decoded according to specific protocol rules to verify their compliance with standards and eliminate anomalous or potentially malicious ones. The traffic then passes through the preprocessing module, which performs low-level operations such as defragmentation and session reconstruction, as well as high-level checks related to application protocols. Once normalized, packets are forwarded to the detection engine, where they are compared with attack patterns using mainly signature-based rules in order to identify malicious behavior. When a match is detected, the alerting module generates and logs an alert and, in IPS mode, can actively block malicious traffic. Finally, the output module concludes the flow by producing statistics and summary information on the analyzed traffic, generally accessible via a command line interface in open source solutions.

Among the NIDPS systems, the most widely-used open source systems [59] are Suricata [34], Snort [60], and Bro/Zeek [61]. Bro/Zeek is merely an IDS, while Suricata and Snort are IPS and IDS because they support both functionalities.

The reasons behind the choice of Suricata for this study are provided in Section 4.

2.3 Detection Paradigms and Scalability

Signature-based systems like Suricata [34] are widely adopted due to their high processing speed, deterministic behavior, and transparent rule sets [34]. Naturally, their structural reliance on predefined signatures makes them highly effective against known threats. However, this same characteristic inherently limits their capability to detect unknown or zero-day attacks. In this context, Neural Networks (NNs)

and other Machine Learning (ML) approaches represent valid and actively researched options. By learning complex, non-linear patterns directly from network traffic, NNs can identify anomalies that deviate from normal baseline behavior, thereby offering a highly effective mechanism to flag zero-day exploits and previously unseen attack vectors [62].

When considering adaptability and scaling, both approaches exhibit distinct and complementary strengths. On one hand, Suricata scales exceptionally well in terms of raw traffic throughput, efficiently handling massive volumes of data through multi-threading [63]. Yet, adapting it to novel threat landscapes requires ongoing, manual rule updates, which can be time-consuming and resource-intensive [64, 65]. On the other hand, NNs offer architectural adaptability: they can be continuously retrained or fine-tuned on newly collected data to seamlessly handle evolving network environments [64]. However, fully leveraging this adaptability requires careful consideration of certain operational constraints. Standard NN deployments can entail higher computational costs [66] and a precise data pre-processing phase is strictly required to properly feed the network packets into the Neural Network. [67]. Furthermore, Neural Networks present challenges related to model interpretability, often referred to as the "black-box" problem [68]. Rather than being an insurmountable barrier, this opacity motivates the development of solutions capable of bridging the advanced detection capabilities of NNs with the strict transparency and auditability required by critical infrastructures, such as IoMT environments.

2.4 Suricata's rule syntax

In signature-based NIDPS architectures, a rule (or signature) represents the fundamental logical unit for threat detection. It formally defines a specific network traffic pattern and instructs the detection engine on the precise action to execute upon identifying a match. Effectively, a rule acts as a deterministic conditional statement that governs the inspection capabilities of the system.

According to the official Suricata documentation [69], rules (or signatures) in Suricata consist of the following parts:

- **Action:** Determines what happens when the signature is matched. Possible actions include passing the traffic (`pass`), dropping the packet and causing a timeout for the sender (`drop`), blocking the traffic and sending a reject packet to the sender and/or recipient (`reject`, `rejectsrc`, `rejectdst`, `rejectboth`), or generating an alert (`alert`).
- **Header:** Defines the protocol, IP addresses (both source and destination), ports (both source and destination), and the rule direction.

- **Protocol**: Specifies the network layer protocol (e.g., `ip`, `icmp`), a transport layer protocol (e.g., `tcp`, `udp`), or an application layer protocol (e.g., `mqtt`, `http`, `dns`, `ssh`, `ftp`).
- **IP addresses**: Can be expressed as a single host, a subnet (using Classless Inter-Domain Routing (CIDR) notation), or a combination of these. A negation or exception operator is supported, alongside predefined variables such as `$HOME_NET` and `$EXTERNAL_NET`.
- **Ports**: Can be expressed as a single port, a range of ports, or a group thereof. The negation or exception operator is similarly applicable for ports.
- **Direction**: Indicates the direction in which the rule is evaluated. It can be unidirectional (`->`) or bidirectional (`<>`). An alternative operator, `=>`, acts like `->` but enables bidirectional matching when used with specific directional keywords. Notably, there is no `<-` operator.

Additionally, the `any` keyword can be used across the protocol, IP addresses, and port fields.

- **Rule options**: Define the specifics of the rule. These keywords instruct the engine on what to search for or how to identify the threat. The most relevant options used in this work are:

- **msg**: A descriptive text string that is logged when the rule triggers, providing a human-readable identifier for the alert (e.g., “Possible IoMT Attack”).
- **flow**: Specifies session tracking properties regarding state and direction. Key arguments include:
 - * **stateless**: Forces the engine to evaluate the packet individually, ignoring the TCP connection state.
 - * **established**: Matches only on packets belonging to a fully open connection (after the handshake).
 - * **to_server** / **to_client**: Restricts detection to a specific traffic direction (request vs. response).
- **dsize**: Matches the packet payload size. It allows the rule to filter traffic based on the data volume, which is often used to detect anomalies (e.g., buffer overflows) by verifying whether the size falls within a specific range.
- **sid**: The Signature ID; a unique numeric identifier for the rule (e.g., 1000001), which is essential for rule management and referencing.

While numerous other rule options exist, those pertinent to this approach are detailed in subsequent sections.

Another key aspect of the syntax is that Suricata enforces a strict architectural separation between packet-scoped and stream-scoped inspection. It explicitly prohibits the commingling of atomic packet keywords (e.g., `dsize`, `flags`, or `ttl`) with stream-reassembled or application-layer keywords within the same signature context [70]. This structural constraint stems from how Suricata processes the network stack: while packet-level keywords are evaluated against individual, stateless units of data, stream-aware keywords require the engine to buffer and reassemble multiple packets to interpret higher-level protocols. Attempting to bridge these two layers within a single rule can lead to logical inconsistencies, as the detection engine may attempt to evaluate packet-level constraints before the full application-layer context is successfully reconstructed [70].

A practical example of a Suricata rule is provided below:

Listing 1 Rule Example in Suricata

```
alert tcp $HOME_NET any -> $EXTERNAL_NET any (msg: "Abnormal
Payload Size"; flow: stateless; dsize: >200;
sid: 1000001;)
```

In this example, the action is highlighted in cyan, the protocol in red, the IP addresses and ports in green, the direction in blue, and the rule options in yellow.

2.5 Suricata for the MQTT Protocol

Although Suricata offers native support for the MQTT protocol through a set of specific keywords designed to inspect protocol headers and payloads [71], these keywords are unnecessarily granular for common threats, such as DoS or Slow DoS attacks [47, 72].

Specifically, while the native engine can inspect individual fields, it lacks the stateful awareness required to correlate multiple messages over time. Many fundamental attacks do not require such a high degree of protocol specialization; instead, they exploit broader behavioral patterns and sequences of interactions between brokers and clients. Consequently, Suricata’s standard implementation remains limited in its ability to autonomously adapt to the dynamic nature of MQTT environments [47]. This leaves a gap that SurIoT aims to bridge by automating the synthesis of higher-level, context-aware rules.

3 Related Work

The rapidly expanding landscape of the IoT has necessitated a paradigm shift in Intrusion Detection Systems (IDS), transitioning from static signature matching toward dynamic, data-driven methodologies. However, the effectiveness of any learning-based IDS is intrinsically bound to the quality and diversity of the underlying traffic data. To address the

critical scarcity of realistic datasets, [73] introduced the *IoT-Flock* framework. Unlike traditional traffic generators, this open-source tool allows for the creation of massive, heterogeneous scenarios by simulating distinct benign and malicious profiles for IoT-specific protocols such as CoAP and MQTT. Experimental results demonstrated that training classifiers like Random Forest on this synthetic yet high-fidelity data could yield detection accuracies exceeding 99.9%, validating that realistic data simulation is a foundational step for robust security. In a similar vein, Zachos et al. [74] proposed a comprehensive IoT/IoMT security testbed designed to bridge the gap between theoretical models and operational deployment. By integrating digital twin technologies via Eclipse Hono and Eclipse Ditto, their testbed facilitates the rigorous stress-testing of Anomaly-based IDS (AIDS) under controlled attack vectors, such as scanning and Denial of Service (DoS), ensuring systems are resilient before real-world deployment.

Building on these foundations, the research community has increasingly explored hybrid architectures to overcome the limitations of single-model detection. Einy et al. [75] presented a hybrid inference system that synergizes the pattern-recognition power of Neural Networks (NN) with the adaptability of Fuzzy Logic. Integrated within the Suricata environment, this approach achieved a notable 96.11% accuracy in mitigating web-based attacks. Nevertheless, the authors acknowledged that the probabilistic nature of fuzzy systems can introduce challenges regarding false positives, particularly in highly dynamic networks. Parallel to probabilistic methods, the authors in [47] proposed a deterministic parsing approach within Suricata, specifically tailored for the MQTT protocol. By enforcing strict protocol compliance checking for anomalies such as missing header fields or abnormal packet lengths, their parser acts as a first line of defense, efficiently discarding malformed traffic and reducing the computational burden on downstream rule-matching engines.

Despite the superior performance metrics of "black-box" Deep Learning (DL) models, their lack of transparency remains a significant barrier to adoption in sensitive sectors like IoMT. Addressing this trust gap, [52] conducted a comparative analysis of Deep Neural Networks (DNN) versus explainable models using synthetic medical traffic. While DNNs provided marginally higher accuracy, the authors emphasized the operational superiority of Fuzzy Hoeffding Decision Trees (FHDT), as they generate linguistically interpretable rules that allow human operators to understand the rationale behind an alert. Preceding recent attempts to automate rule extraction, frameworks such as Syrius [76] focused on rule optimization rather than autonomous generation. By deriving and ranking plausible candidate signatures from an initially provided overspecified seed rule via heuristic data mining, Syrius optimizes existing constraints for general net-

work attacks. However, it does not generate new rules from scratch, nor is it designed for the unique stateless requirements of IoT protocols like MQTT. Addressing the need for transparency, recent advancements have aggressively emphasized Explainable AI (XAI). For instance, Zeghida et al. [77] proposed the XMID-MQTT framework, fundamentally applying SHAP and LIME algorithms to interpret the decisions of a Random Forest classifier against MQTT-specific intrusions. Expanding this interpretability to the physical layer, Georgiades and Hussain [78] introduced a cross-layer XAI framework tailored for the IoMT. Crucially, however, these XAI approaches remain strictly analytical; they explain ML predictions but do not possess the capability to automatically generate deployable Suricata signatures.

Extending the concept of interpretability from analysis to enforcement, Coscia et al. [57] proposed *Anomaly2Sign*; this work leverages Decision Trees (DTs) not just for classification, but as a mechanism to automatically extract deterministic Suricata rules. With F1-scores between 99.7% and 99.9% on DDoS datasets, and supported by an Akaike Information Criterion (AIC) analysis favoring lower-complexity models, they demonstrated the feasibility of converting learned anomalies into rigid, deployable signatures without sacrificing performance.

The state-of-the-art has recently evolved to encompass Generative Artificial Intelligence (GenAI), which promises to automate the labor-intensive process of signature creation. In fact, systematically translating dynamic machine learning insights into static, deterministic enforcement rules is increasingly recognized as a possible, if not necessary, methodology to secure critical infrastructures. Such an approach effectively bridges the gap between the complex predictive power of advanced AI models and the rigid, high-speed execution environments of standard IDS, ensuring that anomaly detection can be operationalized seamlessly. Papoutsis et al. [56] introduced *RuleXploit*, a framework that utilizes GenAI to analyze exploit codes and automatically generate corresponding Suricata rules. By bridging the gap between offensive exploit analysis and defensive rule creation, this approach reduces the reaction time to new vulnerabilities. Similarly, the authors in [55] proposed *Hex2Sign*, a framework employing Large Language Models (LLM) such as GPT-3 and RoBERTa to translate raw hexadecimal packet data into human-readable IDS signatures. Their results, achieving a BERTscore above 96%, highlight the potential of foundational models to interpret complex payload data.

This trend of leveraging Large Language Models has expanded rapidly from simple semantic translation to the synthesis of operational security rules. Moreno and Iturbe [79] demonstrated that off-the-shelf LLMs can generate Suricata rules for Industrial Control Systems without resource-intensive fine-tuning. Pushing this concept further, Li et

al. [80] introduced GRIDAI, a multi-agent LLM framework that generates and automatically repairs existing rulesets to combat adversarial attack variants. However, a critical synthesis of these works reveals a persistent gap: while GenAI models like *RuleXploit*, *Hex2Sign*, and multi-agent systems like GRIDAI offer impressive automation, they often lack the strict determinism required for critical infrastructure protection, and none are targeted at the MQTT protocol. A direct comparison with existing literature is infeasible. To our knowledge, no current study offers a unified approach that synthesizes Suricata rules by combining the deterministic enforceability of explainable tree-based models specifically for the MQTT protocol or broader IoT/IoMT scenarios. Our proposed approach stands strictly apart by performing autonomous, from-scratch signature generation on Suricata explicitly tailored for MQTT IoT environments—a capability currently absent in both existing rule optimizers and ML-based anomaly detectors.

While works like *Anomaly2Sign* [57] successfully validates the transition from ML classification to rigid IDS rules, it remains largely inapplicable to MQTT traffic. Their reliance on stream-scoped features introduces reassembly ambiguity, inherently forcing a generalized view of the network traffic. However, the nature of MQTT in medical and industrial applications relies on atomic, lightweight messages. Therefore, protecting these networks demands a deterministic 1:1 mapping via stateless inspection to pinpoint specific packets accurately. Generalizing data across an entire flow abstracts away the very packet-level details necessary to detect precise injection or formatting anomalies, thus excluding such methodologies from effectively securing most IoT and IoMT environments.

On the other hand, generative approaches like *Hex2Sign* [55] and *RuleXploit* [56] are designed for different operational contexts. Specifically, *Hex2Sign* focuses primarily on semantic translation, mapping payloads to natural language alert labels rather than synthesizing the structural network constraints of a rule from scratch. Similarly, *RuleXploit* relies on the a priori assumption that exploit scripts are available, a condition that is rare in IoT/IoMT environments, where volumetric and zero-day attacks frequently occur without retrievable code. Because these models prioritize linguistic correlation, they differ methodologically from SurIoT, which extracts fully deployable, syntax-compliant rules directly from raw network behavior. Rather than relying on prior exploit scripts, SurIoT employs an operator-guided workflow to tailor the deterministic enforcement logic to specific contextual needs. By synthesizing the core detection constraints independently, SurIoT stands out as a unique approach, effectively bridging the gap between explainable ML and operator-supervised rule enforcement in IoMT/IoT environments.

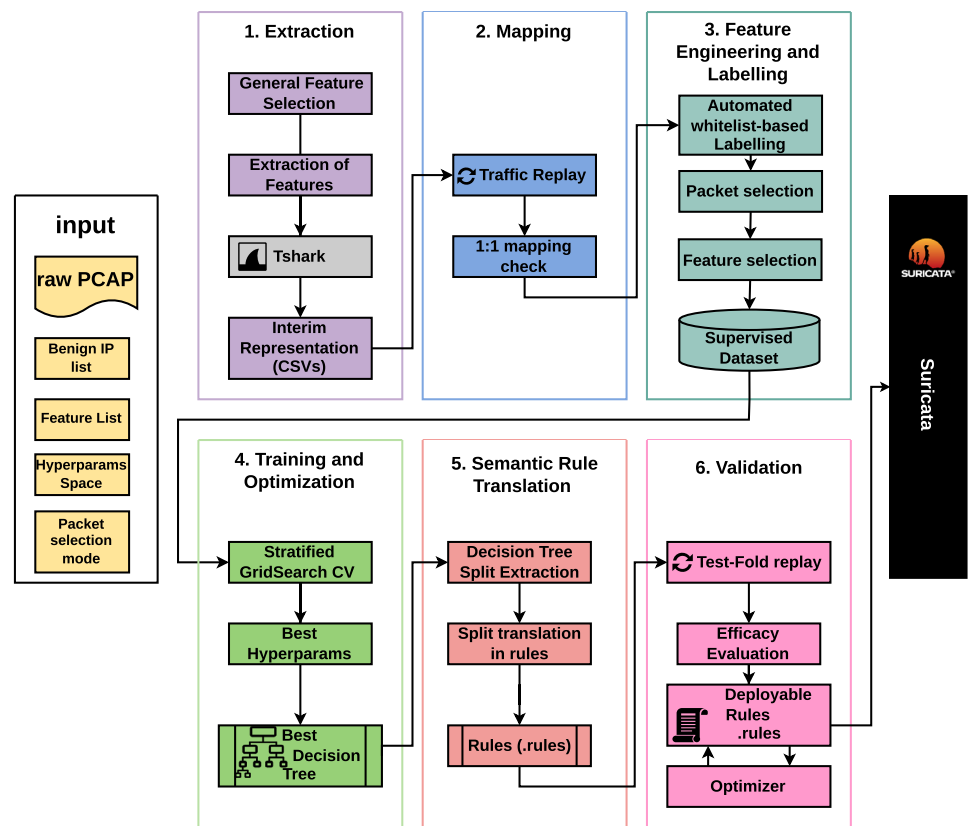
4 Proposed approach: SurIoT

The purpose of this work is to facilitate the effective translation of Machine Learning (ML) models into NIDPS rules for IoT/IoMT environments. To achieve this, we propose SurIoT, an approach designed to automatically synthesize security rules for IoT and IoMT ecosystems by enabling granular interception of MQTT traffic. The objective is to deliver a deployable solution where rules synthesized from empirical results effectively detect and mitigate anomalous behaviors within high-fidelity emulated healthcare scenarios.

Specifically, SurIoT is a rule generator engine that leverages the inherent interpretability of Decision Trees (DTs) to create rules for IoT/IoMT intrusion detection systems (e.g., Suricata [34], Snort [60]). Given a Packet Capture (PCAP) $\mathcal{P}$ containing both benign and malicious traffic, a list of known benign IP addresses $\mathcal{B}$, a hyperparameter space $\mathcal{V}$, a feature list $\mathcal{F}$, and a packet-selection mode $\mathcal{M}$, the SurIoT approach produces a set of rules $\mathcal{R}$ deployable in Suricata [34].

The operational workflow of SurIoT is defined by the following six-step process:

1. **Extraction:** Raw PCAP files are parsed to extract stateless metrics at the IP and TCP layers. We restrict our focus to single-packet features to minimize errors and difficulties during the translation phase.
2. **Mapping:** Prior to training, raw traffic is replayed through a standard IDS configuration equipped with a catch-all rule. This step is needed to establish a one-to-one mapping between packets and alerts.
3. **Feature Engineering and Labeling:** A supervised dataset is constructed using a whitelist-based approach, distinguishing trusted internal hosts from unauthorized external traffic.
4. **Training and Optimization:** The system utilizes DTs to ensure interpretability. Optimization is performed via Stratified Cross-Validation, prioritizing trees with higher scores to prevent overfitting and to guarantee that the resulting logic is concise and easy to translate.
5. **Semantic Rule Translation:** This core phase converts the DT logic into Suricata's [34] rule syntax. It handles the translation of tree path traversals into signatures, consolidates numerical ranges, and establishes a correspondence between leaves and rules.
6. **Validation:** The generated rules are loaded into a clean IDS instance, and the original traffic packets from the test fold of the best model are replayed. The system evaluates its efficacy by measuring the exact correspondence between the IDS alerts and the original ML model predictions. Finally, an additional optimization phase is considered to refine the rules for a production environment.

Fig. 1 General schema of SurIoT

Each of these phases is discussed in detail in the following sections. Additionally, the overall operating schema of SurIoT is illustrated in Figure 1.

SurIoT leverages Suricata [34] as its core NIDPS engine. This architectural choice is supported by recent comparative studies [59, 81, 82], which demonstrate that Suricata outperforms its NIDPS counterparts in terms of attack detection accuracy and computational stability under stress. Although our approach is currently optimized for Suricata, the generated logic remains general and engine-agnostic. The syntax of Suricata shares significant similarities with Snort [60, 69] and other standard engines. Consequently, the decision rules extracted by SurIoT can be adapted to other NIDPS platforms with minimal effort, fostering broader interoperability across heterogeneous security environments.

For the data processing phase, we selected TShark to ensure methodological alignment with the state of the art, where it is widely adopted [50, 51]. Furthermore, its ability to export granular header fields into structured formats such as Comma-Separated Values (CSV) and JavaScript Object Notation (JSON) makes it particularly suitable for automated feature extraction pipelines, guaranteeing both reproducibility and high fidelity in data representation.

Overall, the system is designed to bridge the gap between ML capabilities and rule-based efficiency. The choice of

using DTs as the source model for rule generation is motivated by three fundamental reasons:

- First, several recent studies [37, 83–85] have shown that tree-based models are particularly effective in classifying IoMT and IoT traffic, as they handle the specific feature distributions of these environments efficiently;
- Second, DTs are intrinsically explainable [52, 86–88]. This “white-box” nature allows human operators to understand and verify the logic leading to a classification, ensuring that the resulting rules are transparent and auditable;
- Third, as highlighted in [57], the hierarchical structure of a decision tree is semantically compatible with the *if-then-else* logic of standard NIDPS rules, providing an optimal starting point for automated translation.

It is important to note that the choice of Decision Trees is not merely a preference but an architectural constraint imposed by the rule translation requirement. Alternative classifiers were considered and systematically excluded based on their incompatibility with deterministic rule synthesis:

- **Ensemble methods** (e.g., Random Forest, XGBoost, Gradient Boosting): These models aggregate predictions from hundreds of weak learners through majority vot-

ing or additive boosting. The resulting decision logic is distributed across the entire ensemble, and no single tree captures the complete classification boundary. Suricata's rule engine evaluates signatures independently; it has no mechanism to implement inter-rule voting or weighted aggregation. Translating an ensemble would produce hundreds of conflicting rules, with no way to express the consensus logic that determines the final prediction. While individual trees from an ensemble could theoretically be extracted, doing so would discard the very aggregation mechanism that provides their performance advantage, yielding rules that are individually weaker than those derived from a single, purpose-trained Decision Tree.

- **Hyperplane-based models** (e.g., SVM, Logistic Regression): These classifiers partition the feature space using continuous linear or nonlinear decision boundaries of the form $\mathbf{w}^T \mathbf{x} + b = 0$. Suricata's rule keywords operate on discrete, per-feature thresholds (e.g., `ttl:>64`; `dsize:100-500`), which correspond to axis-aligned splits. Expressing an oblique hyperplane that combines multiple features into a single linear inequality is not supported by the rule syntax, making lossless translation impossible.
- **Neural Networks and Deep Learning:** Beyond the opacity concerns already discussed, these models encode decision logic in distributed weight matrices across multiple layers. No known method exists to extract deterministic, per-packet Suricata rules from such representations without substantial approximation loss.

Consequently, the Decision Tree remains the *only* classifier family for which every root-to-leaf path maps bijectively to a syntactically valid Suricata signature, preserving the learned decision boundary without approximation.

A critical advantage of the SurIoT approach is the decoupling of analysis and enforcement. While the DT algorithm is computationally intensive, it is executed only once during the offline training phase to analyze traffic patterns and extract the underlying logic. Once the tree is translated into Suricata signatures, the system performs classification in real-time at the packet level, ensuring negligible latency. This strategy overcomes the traditional bottlenecks [89, 90] associated with deploying heavy ML models directly in the data path, effectively combining the “intelligence” of ML with the speed of a signature-based engine.

The rule generation process is highly parameterized to ensure flexibility across diverse deployment scenarios. First, the system accepts the desired enforcement *action* as a parameter. For instance, although our experimental validation (see Section 5) utilized the `alert` action to measure detection metrics, the system can readily generate `drop` or

`reject` rules to actively block threats. Second, SurIoT supports the injection of custom *rule options* (e.g., specific `msg` fields or flag bits) into the generated signatures to enhance logging and event correlation.

4.1 Extraction

The extraction phase is defined as follows: given a set of input PCAP files and a predefined set of target features, Tshark is employed to parse the raw traffic data. Tshark extracts key metadata specifically from the Network (IP) and Transport (TCP) layers. The focus is strictly placed on *stateless, per-packet features*, such as Time-to-Live (TTL), Window Size, Sequence Numbers, and Flag states.

This restriction is a *correctness requirement* rooted in a fundamental architectural constraint: the classifier and the enforcement engine *must* observe the same data representation for the translated rules to be semantically faithful. Suricata's detection engine internally classifies signatures into distinct types `pkt` (packet-level), `stream` (reassembled stream), and `app_layer` (application protocol), each operating on a distinct and often incompatible view of the network traffic [91].

Stateful rule types do not inspect raw packets; instead, they match against data reconstructed by Suricata's stream engine, which reassembles TCP segments into an ordered byte stream [92]. Consequently, the dynamic reconstruction of TCP sessions introduces an intrinsic non-determinism between offline feature extraction tools and live enforcement sensors. This fundamental divergence makes the translation of stateful, flow-based machine learning models into deterministic rules not merely complex, but mathematically and architecturally impossible. Because an idealized offline parser and a memory-constrained live stream engine resolve sequence overlapping and evasion ambiguities differently, establishing formal observational equivalence between the two operational environments is structurally unachievable.

Per-packet header fields, by contrast, bypass these reassembly layers entirely. Keywords such as `ttl`, `window`, `tcp.flags`, `seq`, and `ack` inspect the original, unaltered header values of each packet exactly as they appear on the wire [93]. This guarantees *observational equivalence*: every split learned by the decision tree maps to a Suricata rule condition that evaluates the identical numeric value on the identical packet.

Beyond ensuring formal correctness, this approach provides significant operational advantages. By maintaining a strict one-to-one semantic mapping, each tree split corresponds directly to a specific rule keyword, comparison operator, and threshold, as detailed in Section 4.5. Furthermore, this stateless paradigm drastically reduces computational overhead by obviating the need for deep flow reassembly and state tracking. Since operations on isolated packets are inher-

ently less resource-intensive than maintaining the complex state machines required for flow-based analysis, the system can more effectively sustain wire-speed processing in high-throughput IoT and IoMT environments.

4.2 Mapping

In the Mapping phase, a correspondence is established to ensure that for every alert generated by Suricata, a corresponding packet is identified by TShark. Formally, given the set of alerts $\mathcal{A}$ and the set of records produced by TShark $\mathcal{P}$, the mapping phase defines a function $\mathcal{T} : \mathcal{A} \rightarrow \mathcal{P}$ that uniquely maps each alert to its corresponding packet.

The Mapping phase is pivotal for ensuring operational consistency between the Machine Learning (ML) model and the IDS engine. For the classification results to be valid, the feature extraction pipeline and the verification engine must share a congruent view of the data. This alignment requires two strict conditions:

1. **Packet Consistency:** Both systems must process an identical set of packets.
2. **Structural Alignment:** Both systems must perceive the same structural attributes (e.g., indices and header values) for each individual packet.

To ensure pipeline integrity, a configuration utilizing a “catch-all” signature designed to intercept all TCP traffic is adopted:

Listing 2 Atomic Packet Catch-all Rule

```
alert tcp-pkt any any -> any any (msg:"pkt"; sid:1000001;
    flow:stateless; rev:1;)
```

The selection of the `tcp-pkt` protocol identifier in Listing 2 is a fundamental decision. While standard documentation [70] often presents protocol options interchangeably, empirical analysis [94] reveals distinct inspection behaviors that critically affect data alignment:

- **tcp:** This option activates a hybrid inspection mode where the engine performs detection on both individual packets and reassembled streams. In scenarios where detection overlaps, Suricata prioritizes stream results. This behavior introduces ambiguity in the alert output and, crucially, in the packet index (`pcap_cnt`), making a deterministic 1:1 mapping with the feature vector nearly impossible.
- **tcp-stream:** This restricts matching exclusively to the reassembled TCP buffer. Consequently, alerts are often attributed to the final packet of a sequence (e.g., a FIN packet) or a buffer flush event, rather than the specific packet containing the payload. This causes a misalign-

ment with the feature extraction phase, which operates on discrete frames before reassembly.

- **tcp-pkt:** This option forces the engine to inspect strictly at the Protocol Data Unit (PDU) level, bypassing the reassembly logic.

By enforcing `tcp-pkt`, the packet index returned by the IDS corresponds exactly to the index observed in the raw PCAP file (consistent with Wireshark/Tshark), thereby guaranteeing the required structural alignment.

4.2.1 Keyword Scope Constraints

A critical operational constraint in Suricata is the strict difference between packet-scoped and stream-scoped inspection. Unlike other detection engines (e.g., Snort [60]) which may tolerate hybrid rule definitions, Suricata explicitly prohibits the commingling of atomic packet keywords such as `dsizes`, `flags`, and `ttl` with stream-reassembled keywords (e.g., `http_*`) [70].

This separation is enforced to preserve the determinism of the detection logic. Empirical evidence [94] suggests that mixing scopes can lead to undefined behavior; specifically, introducing packet-level constraints like `dsizes` into a stream-aware rule can preemptively trigger evaluation before the application-layer protocol is fully identified [70]. This race condition typically results in detection failures on the initial packets of a flow (false negatives), as protocol metadata is not yet available during the packet-level check. Consequently, the rule generation methodology rigorously adheres to this constraint by utilizing exclusively stateless keywords, ensuring full compatibility with the `tcp-pkt` inspection mode.

4.3 Feature Engineering and Labeling

In the feature engineering and labeling phase, the dataset undergoes a pre-processing stage designed to ensure data quality and relevance. This workflow facilitates the construction of a robust model by removing noise and establishing a reliable ground truth. The process consists of three sequential steps:

1. **Automated Whitelist-based Labeling:** To establish a reliable ground truth without manual intervention, a whitelist approach based on the known network topology is adopted. Specifically, traffic originating from source IP addresses included in the whitelist is classified as benign, whereas traffic from unlisted sources is labeled as malicious. This logic ensures that all traffic not explicitly originating from verified nodes is treated as hostile. Consequently, this pipeline eliminates the subjectivity of

manual tagging and guarantees that the “Attack” class captures all external or unauthorized injection attempts. This strategy is particularly effective in IoMT scenarios where the internal network topology (e.g., when using MQTT) is static or easily recoverable.

2. **Packet Selection:** The dataset is filtered according to the specific attack mode $\mathcal{M}$ selected by the user. This step ensures that only relevant packets are retained for analysis:

- If $\mathcal{M}$ is volumetric (e.g., DoS, Slow DoS), packets without a payload are retained, as they contribute to the overall attack volume.
- If $\mathcal{M}$ is non-volumetric (e.g., buffer overflow, malformed data), empty packets are excluded, since detection relies on analyzing malicious payload content.

3. **Feature Selection:** In the final phase, irrelevant or noisy features are eliminated to prevent the model from learning spurious correlations. Specifically, the following fields are removed by default:

- **ip.src** and **ip.dst**: These addresses are considered noise. Their inclusion would lead the model to memorize the specific network topology rather than learning generalizable packet characteristics.
- **frame.number**: This sequential index assigned by TShark lacks semantic value. Its inclusion introduces spurious correlations, hindering the identification of robust patterns.
- **tcp.window**: This field reflects the specific operating system stack and dynamic congestion states. It introduces an environmental bias, causing the model to fingerprint specific hardware configurations rather than abstract attack behaviors.
- **tcp.ack** and **tcp.seq**: These cumulative counters depend on the specific connection history. Their inclusion causes overfitting, as the model would memorize the unique state of a single stream instead of learning abstract traffic signatures.

Finally, the input is further processed to retain only the specific feature set $\mathcal{F}$ defined by the operator. This ensures the model focuses exclusively on the most relevant attributes for the analyzed traffic type. Retaining irrelevant features could lead to the generation of unnecessarily complex rules, or rules that overfit benign traffic specifics rather than the actual attack structure.

4.4 Model Training and Optimization

In the training and optimization phase, the DT model is trained using the optimal configuration identified via a grid search over the hyperparameter space.

4.4.1 Hyperparameter Optimization

Formally, a hyperparameter θ is a pre-defined configuration variable external to the learned model parameters ϕ . While ϕ is estimated from the training data $\mathcal{D}$ to minimize a loss function $\mathcal{L}$, the hyperparameters $\theta \in \Theta$ control the capacity of the learning algorithm $\mathcal{A}$ and its regularization constraints.

Given that the model $\mathcal{M}$ is a Decision Tree (DT), optimizing θ is essential to maximize both generalization capabilities and computational efficiency, thereby minimizing the generalization error E_{gen} . To achieve this, we conducted a grid search over the hyperparameter space Θ . The selected hyperparameters and their specific relevance to the NIDPS domain are detailed below:

- **max_depth**: Constrains the maximum vertical growth of the tree. While deeper trees often achieve higher training accuracy by capturing finer details, they are highly prone to overfitting. In the context of intrusion detection, limiting this depth is crucial to enforce model generalization. This ensures that the generated rules capture robust traffic patterns rather than overspecializing on noise or specific artifacts within the training set.
- **min_samples_split**: Defines the minimum number of samples required to split an internal node. This threshold prevents the algorithm from creating decision rules based on statistically insignificant subsets of packets, ensuring that splits represent robust traffic patterns rather than isolated anomalies or noise.
- **min_samples_leaf**: Sets the lower bound for the number of samples in a terminal node. Enforcing a minimum cluster size at the leaf level smooths decision boundaries, effectively reducing the model's sensitivity to outliers and ensuring that no class is predicted based on a negligible number of instances.
- **criterion**: Specifies the function H used to evaluate the quality of a split.
- **random_state**: Initializes the pseudo-random number generator used for feature selection at split points. This parameter guarantees deterministic behavior, ensuring that the training process of $\mathcal{M}$ yields fully reproducible results across different experimental runs.

4.4.2 Grid Search with Stratified K-Fold Cross-Validation

The hyperparameter optimization process employs an exhaustive grid search strategy designed to identify the configuration yielding the highest generalization performance. The optimization objective is specifically formulated to address the inherent class imbalance often observed in IoMT anomaly detection [83, 95, 96], where the distribution of benign and malicious instances is rarely uniform.

Consequently, the selection process prioritizes a configuration that achieves a balanced trade-off between system sensitivity (minimizing missed detections) and prediction reliability (limiting false alarms), rather than merely maximizing raw accuracy.

To further ensure statistical robustness, the grid search is coupled with Stratified K-Fold Cross-Validation. Unlike standard random splitting, this technique guarantees that the proportion of samples for each class remains consistent across every fold. This ensures that the model is trained and validated on representative subsets of the data, thereby preventing performance assessments from being skewed by random variations in class distribution during the validation phase.

4.5 Semantic Rule Translation

This component bridges the gap between the ML model and the NIDPS by translating the DT's logical predicates into syntactically compliant Suricata signatures. The rule extraction process is exhaustive and class-agnostic, generating candidate rules for both *benign* and *malicious* traffic flows. This approach avoids enforcing rigid filtering policies; instead, it provides a comprehensive output, granting operators the flexibility to select and deploy the ruleset that best aligns with their specific operational needs.

4.5.1 Translation of Integer Variables

A DT partitions the feature space using binary inequalities. However, traversing a path often results in multiple redundant constraints on the same feature. The translation logic must therefore aggregate these disjointed constraints into a single, definitive validity interval.

A critical semantic adaptation is required to align the decision boundaries with the specific range syntax adopted by the target engine. Since Suricata ranges are interpreted as *exclusive* (i.e., matching values strictly between the bounds) [97], the translation algorithm must expand the boundaries of the logical interval derived from the tree. For example, if the DT isolates a traffic class within the inclusive interval [20, 500], mapping this directly would erroneously exclude the boundary values. To preserve the model's exact logic, the algorithm must decrement the lower bound and increment the upper

bound. Formally, to capture the set $\{x \in \mathbb{Z} \mid 20 \leq x \leq 500\}$, the system generates the signature format [19:501]. This ensures that the exclusive check ($19 < x < 501$) correctly validates all integers from 20 to 500.

The formal procedure for this normalization is detailed in Algorithm 1.

Algorithm 1 Continuous Feature Interval Normalization

Require: List of path constraints $\mathcal{C}_f$ for feature f

Ensure: Normalized Range String ρ

```

1:  $v_{min} \leftarrow 0$ 
2:  $v_{max} \leftarrow \infty$ 
3: for all  $(operator, value) \in \mathcal{C}_f$  do
4:   if  $operator$  is  $(\leq)$  then
5:      $v_{max} \leftarrow \min(v_{max}, value + 1)$ 
6:   else if  $operator$  is  $(>)$  then
7:      $v_{min} \leftarrow \max(v_{min}, value)$ 
8:   else if  $operator$  is  $(\geq)$  then
9:      $v_{min} \leftarrow \max(v_{min}, value - 1)$ 
10:  end if
11: end for
12: if  $v_{min} > 0$  and  $v_{max} < \infty$  then
13:   return format " $[v_{min} : v_{max}]$ "
14: else if  $v_{min} > 0$  then
15:   return format " $> v_{min}$ "
16: else if  $v_{max} < \infty$  then
17:   return format " $< v_{max}$ "
18: end if
19: return  $\emptyset$ 

```

4.5.2 Bitwise Logic and Flag Adaptation

The translation of protocol flags involves a fundamental change in representation. The DT treats each TCP flag as an independent binary feature. Conversely, Suricata evaluates flags as a single 8-bit field, utilizing a mask-based syntax: `flags: <matches>, <ignore_mask>`.

To ensure the generated signatures strictly adhere to the model's decision path, the translation algorithm employs a ternary classification for the set of all possible flags Ω_{flags} :

1. **Required Set** (Φ_{set}): Flags that the tree explicitly checks as equal to 1.
2. **Required Unset** (Φ_{unset}): Flags that the tree explicitly checks as equal to 0.
3. **Irrelevant** (Φ_{ignore}): Flags that do not appear in the current decision path.

The system constructs the rule by listing Φ_{set} as the match criteria and computing the mask as the union of all irrelevant flags (Φ_{ignore}). This logic aligns with Suricata's capability to handle complex session initiation packets, such as Explicit Congestion Notification (ECN) scenarios. As described in the official documentation [98], a rule checking for a SYN

packet while ignoring the values of reserved bits (like Congestion Window Reduced (CWR) and ECN-Echo (ECE)) is expressed as `tcp.flags:S,CE`. In our methodology, this corresponds to a decision path where SYN is required to be 1, CWR and ECE are not evaluated, and all other flags are implicitly required to be 0. Algorithm 2 formalizes this transformation, while Figure 2 provides a visual explanation of the proposed algorithm.

Algorithm 2 TCP Flag Bitmask Construction

Require: List of path constraints $\mathcal{C}_{flags}$ regarding TCP bits

Ensure: Suricata Flags String σ

```

1:  $\Omega_{all} \leftarrow \{F, S, R, P, A, U, C, E\}$   $\triangleright$  set of possible TCP flags
   keyword in Suricata
2:  $\Phi_{set} \leftarrow \emptyset$ 
3:  $\Phi_{unset} \leftarrow \emptyset$ 
4: for all  $(flag, state) \in \mathcal{C}_{flags}$  do
5:   if  $state == 1$  then
6:      $\Phi_{set} \leftarrow \Phi_{set} \cup \{flag\}$ 
7:   else
8:      $\Phi_{unset} \leftarrow \Phi_{unset} \cup \{flag\}$ 
9:   end if
10: end for
11:  $\Phi_{ignore} \leftarrow \Omega_{all} \setminus (\Phi_{set} \cup \Phi_{unset})$ 
12:  $str\_match \leftarrow \text{Concatenate}(\Phi_{set})$ 
13:  $str\_mask \leftarrow \text{Concatenate}(\Phi_{ignore})$ 
14: if  $str\_match \neq \emptyset$  then
15:   return "flags: " +  $str\_match$  + ", " +  $str\_mask$ 
16: else
17:   return "flags: 0, " +  $str\_mask$ 
18: end if

```

4.6 Validation

The validation phase is critical for verifying the operational fidelity of the translated Suricata signatures against the source ML model. To ensure a rigorous assessment and prevent data leakage, this process is conducted within the Stratified k -Fold approach utilized during the optimization phase.

Let Θ^* denote the optimal set of hyperparameters identified during the grid search. To validate the translation logic, a specific fold k_i is randomly selected from the dataset partitions. The DT model $\mathcal{M}$ is re-trained on the training split $\mathcal{D}_{train}^{(i)}$ using the configuration Θ^* , and the resulting decision paths are translated into a set of Suricata signatures $\mathcal{R}$.

The validation methodology employs a dual-stream execution strategy on the held-out test split $\mathcal{D}_{test}^{(i)}$. This ensures that the evaluation is performed on unseen traffic, simulating a real-world deployment scenario:

1. **Model Inference:** The trained model $\mathcal{M}$ generates class predictions Y_{ML} for each data instance p contained within the test subset $\mathcal{D}_{test}^{(i)}$.
2. **Suricata Validation:** The network traffic capture is replayed through the Suricata engine, configured with the

synthesized ruleset $\mathcal{R}$. Following the replay, the resulting logs (stored in `eve.json`) are parsed to identify and extract only the specific entries that correspond to the packets constituting the test set, ensuring a consistent comparison baseline.

In our approach, the alignment between Suricata's alerts and the tree's predictions is structurally guaranteed by the design of the preprocessing pipeline. The entire feature extraction process was engineered to synchronize the TShark view with the Suricata inspection stream, explicitly associating the TShark `frame.number` with the corresponding Suricata `pcap_cnt` identifier at the dataset generation stage.

Crucially, to ensure the immediate recognition of the classification outcome during the replay phase, the predicted class label is explicitly encoded into the `msg` metadata field of each generated signature (e.g., `msg: "decision_tree for class X"`), unless `msg` is custom-specified by the user. Following this alignment, a formal verification is performed. The test traffic is processed in parallel: the theoretical model generates predictions based on the features extracted by TShark, while the Suricata engine inspects the packets using the translated rules. A detection is confirmed valid only if the alert logged in `eve.json` matches the packet index and the content of the `msg` field corresponds to the expected class. These operational detections are then compared directly against the model's baseline to assess whether the ruleset faithfully replicates the decision boundaries without semantic loss.

4.6.1 Optimization

Beyond the generation of classification boundaries, the SurIoT approach includes a dedicated optimization phase designed to tailor the resulting decision logic to specific operational requirements. This stage bridges the gap between theoretical detection and practical enforcement, relying on explicit input from the operator to transform raw statistical rules into actionable defense policies.

The optimization process is driven by the user to address two critical aspects of deployment:

- **Action Definition (IDS vs. IPS):** The operator determines the appropriate response strategy by specifying the desired rule action. The approach allows the user to choose between a passive `alert`, which maintains a monitoring posture typical of an IDS, or an active `drop` directive. This selection empowers the system to transition into an IPS, allowing the real-time neutralization of malicious packets at the network interface.

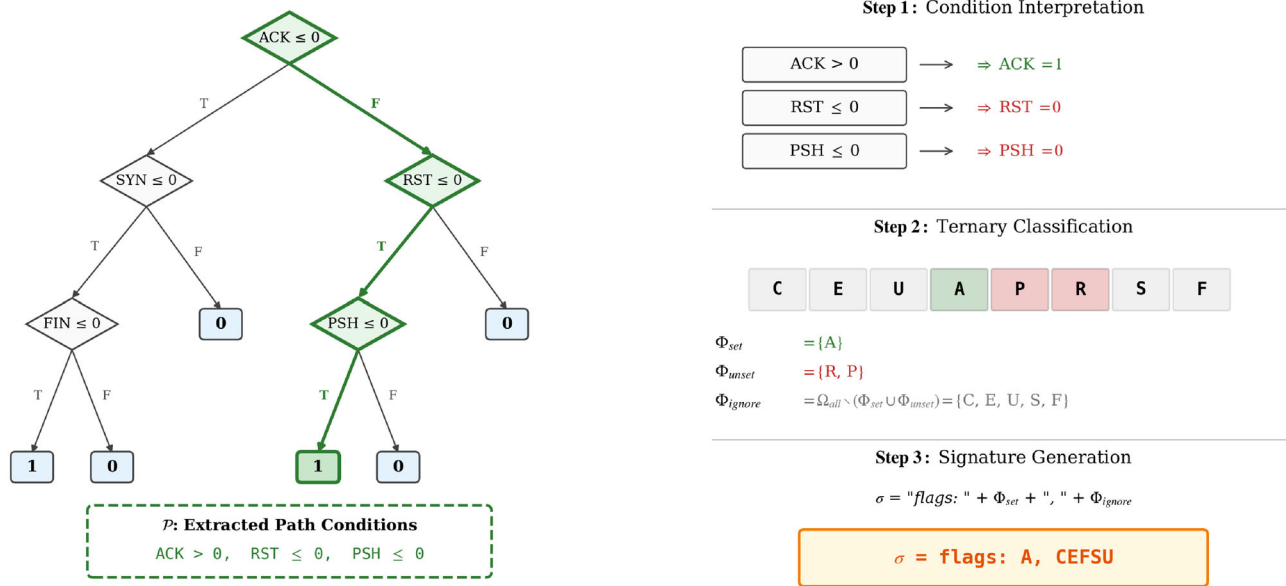


Fig. 2 Flag translation example. Left: A simple DT. Right: Details of the translation process for the selected green path

- Contextual Adaptation:** To ensure seamless integration within specific network topologies, the operator provides the necessary environmental variables. This includes defining the directionality of traffic (e.g., \$EXTERNAL_NET to \$HOME_NET), restricting the rule's scope to specific service ports or subnets (e.g., \$MQTT_PORTS), and modifying other parameters (such as msg). By embedding these context-aware constraints, SurIoT ensures that the generated logic is strictly bound to the intended infrastructure, thereby reducing the risk of false positives affecting unrelated traffic, that can imply an alert fatigue attack.

At the conclusion of this phase, the user obtains the final ruleset for both benign and malicious traffic.

5 Experiments

This section details the experimental evaluation conducted to validate the SurIoT approach. The primary objective is to assess the effectiveness of the synthesized Suricata rules, both in terms of classification metrics and processing, by comparing them against standard Machine Learning (ML) pipelines operating on raw network traffic.

As noted in Section 3, since SurIoT is the first system that extends Suricata's capabilities to MQTT traffic through automated rule synthesis, there are no directly equivalent systems in the literature. Therefore, the approach is benchmarked against robust baseline models. The overall workflow of the evaluation methodology is illustrated in Figure 3.

The remainder of this section is organized as follows: Section 5.1 details the experimental setup, 5.2 details the datasets chosen for the experiments, and Section 5.3 reports the feature engineering phase. Section 5.4 shows the models selected for the comparative analysis. Section 5.5 defines the validation metrics. Section 5.6 reports and discusses the experimental results, while Section 5.7 analyzes the explainability and operational deployment of the generated rules. Finally, Section 5.8 discusses robustness considerations and limitations.

5.1 Experimental Setup

To properly evaluate the approach, it is necessary to establish a baseline pipeline. It should be noted that in a real-world scenario, unlike the native Suricata engine, a Decision Tree (DT) or any other ML model cannot be used directly on raw network traffic, but inherently requires an intermediate traffic processing phase.

This baseline pipeline consists of the following sequential phases:

- (i) Extraction & Selection:** Raw PCAPs are processed by TShark to extract the selected packet-level features.
- (ii) Feature Pre-processing:** To align the extracted features with the ML/DL logic, a casting and alignment phase is conducted (e.g., TShark flags are extracted in hexadecimal format; therefore, a translation is needed).
- (iii) ML/DL Classification & Suricata/Snort Testing:** The ML/DL models are used to classify the pre-processed traffic, while Suricata and Snort (rule translated from

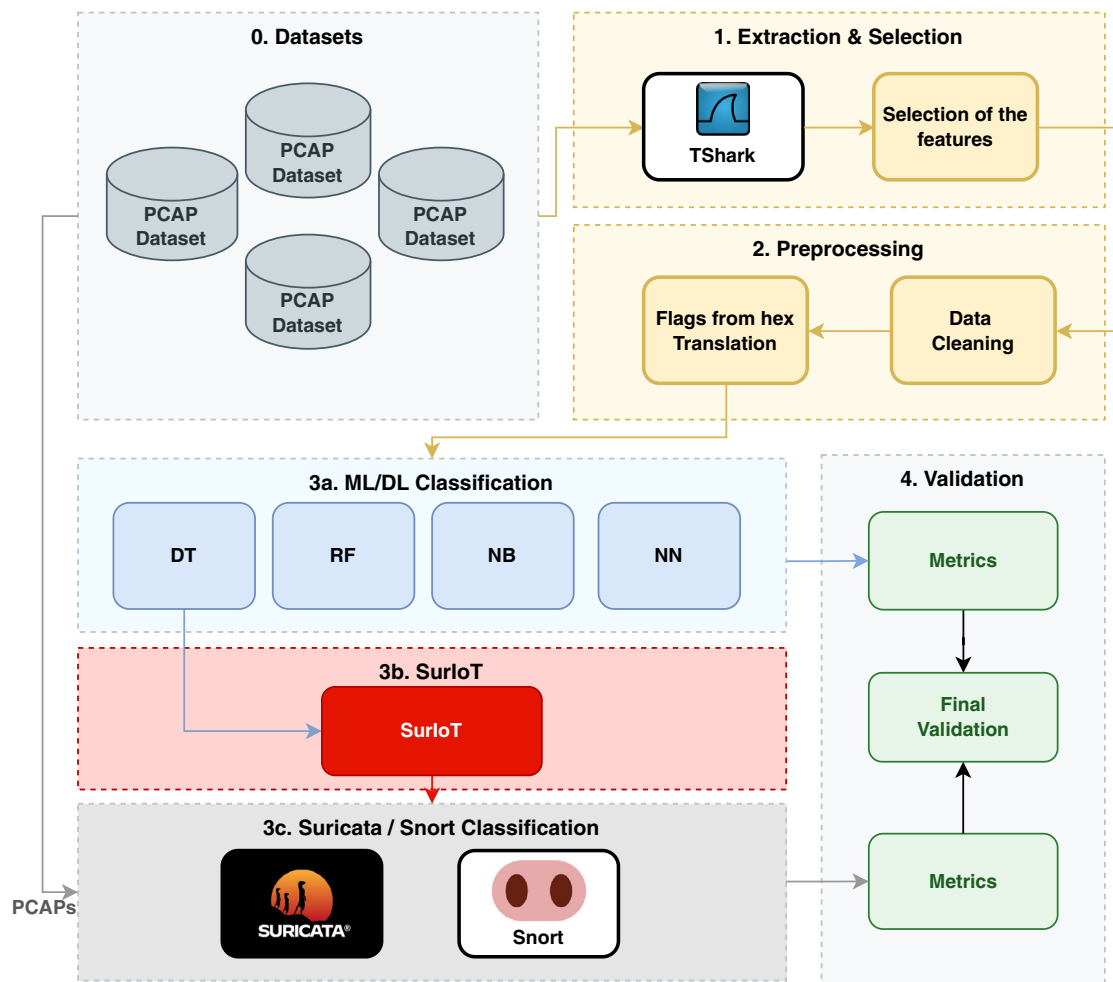


Fig. 3 Pipeline of the performed experiments

Suricata[99]) are independently fed with the synthesized rules obtained from the DT and applied directly on the raw PCAPs.

- (iv) **Validation:** Suricata and the reference ML/DL models are compared to guarantee a perfect 1:1 behavioral mapping through the Hold-out set.

5.1.1 Hardware Setup & Software Dependencies

The workstation employed for the experiments is equipped with an Intel Core i7-8700K processor, 64 GB of RAM, and an NVIDIA GeForce RTX 4060 Ti GPU with 16 GB VRAM.

- The Suricata version used in this work is 8.0.2.
- The TShark version used in this work is 4.2.2.
- The Snort version used in this work is 2.9.

5.2 Datasets

To validate the proposed methodology, a review of existing literature was conducted to identify suitable datasets. However, several widely-used datasets were determined to be incompatible with the specific technical constraints and objectives of this study. The following datasets were analyzed and excluded for specific technical reasons:

- Wustl-EHMS [96] and ECU-IoHT [95]: They were excluded due to the unavailability of raw PCAP files, which are strictly necessary for the deep packet inspection and traffic replay mechanisms required by the proposed system.
- CICIoMT2024 [83]: The absence of clear documentation concerning pre-processing applied to the raw captures prevents the reliable ingestion of this data into the pipeline. To ensure the integrity of the input features and avoid introducing unknown artifacts, this dataset was excluded from the experimental scope.

During this review, the *IoMT Traffic Data* dataset [100] was identified as highly suitable for our methodology, as it provides the necessary raw traffic captures. Crucially, the packets within these captures were preserved in their exact chronological sequence, without any ambiguous mixing or artificial reordering. Furthermore, the dataset is supported by comprehensive and unambiguous documentation, readily accessible via its official GitHub repository [101]. The only required preprocessing step was the exclusion of two specific Layer 2 (L2) attacks. This filtering was strictly necessary because our proposed approach explicitly focuses on Layer 4 (L4) network behaviors.

In addition to this publicly available dataset, and to ensure a fully controlled evaluation environment, synthetic datasets were also generated to guarantee full compatibility with the detection pipeline, employing IoT-Flock [73] and a penetration software for MQTT called *mqttsa* [102].

5.2.1 Overview of IoMT-TrafficData

The *IoMT-TrafficData* dataset [100] was specifically designed to address the scarcity of representative, high-quality data for training Machine Learning (ML) models in healthcare environments. It establishes a robust benchmark for evaluating Intrusion Detection Systems (IDS) by providing a realistic network topology and a comprehensive set of attack vectors tailored to the Internet of Medical Things (IoMT).

To generate the dataset, the authors simulated a highly realistic sports clinic scenario. For the scope of our research, we specifically focused on the wired, IP-based infrastructure of the testbed. The environment featured a General IoT Area and an Adversary Area interconnected via an Ethernet switch and router. The network incorporated medical and environmental devices, such as motion sensors and temperature/humidity sensors (DHT11), connected to core processing nodes (e.g., Arduino Yún and Raspberry Pi). Crucially, the communication across this Ethernet infrastructure utilised standard IoT and IoMT application protocols, specifically MQTT, CoAP, and HTTP, ensuring high fidelity to real-world medical deployments.

The dataset captures both benign traffic and malicious activities generated through distinct cyberattacks. The threat landscape modelled in the dataset is particularly relevant for network inspection and is structured into the following specific attack vectors:

- **Denial of Service (DoS) attacks:** Aimed primarily at disrupting the availability of the central servers by exhausting their memory or concurrent connection pools. The dataset implements the following variations:

- **Apache Killer:** Exploits the HTTP Range header to exhaust server resources by requesting multiple overlapping data ranges [103].
- **R-U-Dead-Yet (RUDY):** A slow-rate attack that targets web applications by submitting form data at an extremely slow pace, keeping the server connections open indefinitely [103].
- **Slow Read:** The attacker sends legitimate requests but reads the server's responses very slowly, tying up concurrent connection slots [103].
- **Slow Loris:** Keeps HTTP connections open by sending partial request headers periodically, eventually exhausting the server's maximum connection pool [103].

- **Application-layer attacks:**

- **MQTT Malaria:** Floods the MQTT broker with an overwhelming volume of illegitimate publish/subscribe requests, disrupting critical communications between medical sensors and back-end systems [104].

- **Reconnaissance attacks:**

- **Network Scanning:** Conducts broad scanning to map the IoMT ecosystem, identify active hosts, and discover open ports or vulnerabilities [105].

- **Network and Protocol-specific attacks (Layer 2):**

- **ARP Spoofing:** Falsifies MAC addresses to intercept, manipulate, or redirect network traffic [106].
- **CAMTableOverflow:** Inundates a network switch with an excessive number of counterfeit MAC addresses, forcing the switch into hub mode to broadcast traffic to all ports [107].

From a structural perspective, the IP-based Ethernet captures yielded approximately 3 million packets of benign traffic and over 15.3 million packets of malicious traffic. A significant advantage of the *IoMT-TrafficData* dataset is its provision of raw, packet-oriented data (PCAP files) alongside robust, chronological documentation.

It is important to note that the current methodology of SurIoT relies entirely on Layer 4 (L4) stateless features such as TCP flags, port distributions, and payload sizes to synthesize its decision logic. Consequently, while attacks like Network Scanning and DoS were successfully retained and processed, the purely Layer 2 (L2) attacks present in the dataset (specifically ARP Spoofing and CAM Table Overflow) were explicitly filtered out during the pre-processing phase. Their detection requires MAC-layer inspection rather than transport-layer dynamics.

5.2.2 Overview of Synthetic Datasets

While the publicly available *IoMT-TrafficData* corpus provides a robust, heterogeneous foundation for evaluating generalized real-world performance. Synthetic scenarios are meticulously designed to evaluate the approach behavior under strictly controlled conditions, allowing for a granular analysis of how the system responds to isolated volumetric exhaustion and stealthy payload anomalies. The following sections detail the methodology employed to simulate the baseline network and inject targeted malicious traffic.

Synthetic Dataset Creation

The dataset was generated by simulating a realistic health-care environment, specifically an Intensive Care Unit (ICU), where critical medical devices communicate via the MQTT protocol. The network infrastructure relied on Mosquitto [108], configured without authentication to simulate a vulnerable deployment scenario.

The two tools used to generate benign and malicious traffic are:

- **IoTFlock** [73]: An open-source framework designed for the generation of massive-scale, realistic IoT traffic. It allows for the definition of specific IoT use cases (e.g., Smart Home, Healthcare) and supports standard protocols such as MQTT and CoAP. In this context, it was utilized to simulate the baseline network activity, generating both benign traffic and malicious patterns to replicate a heterogeneous environment.
- **mqttsa** [102]: A specialized penetration testing tool tailored specifically for the MQTT protocol. It is designed to audit MQTT brokers by performing reconnaissance, brute-force attacks, and Denial-of-Service (DoS) attempts. It was employed as a targeted attack vector generator to inject specific, sophisticated vulnerabilities into the dataset.

Benign Traffic Generation

Using IoTFlock [52, 73], a simulated network of medical sensors and actuators publishing telemetry data in JSON format is created. The scenario included devices such as heart monitors, ventilators, and infusion pumps. Each device was configured with specific IP addresses, MQTT topics, QoS levels, payload ranges to mimic real-world fluctuations (e.g., heart rate between 80-85 bpm), and time profiles. Table 2 summarizes the configuration of the benign nodes.

Malicious Traffic Generation

To introduce malicious behavior, the *mqttsa* tool [102] was employed, following the methodology previously outlined. This tool allowed for the execution of targeted attacks against the broker to generate labeled anomalies within the dataset. The attack vectors are:

Table 2 Configuration of Simulated Benign IoT Devices (ICU Scenario)

Device Name	Topic	Payload	QoS	Interval
Bed1-HeartMonitor	hosp/icu/bed1/cardio/bpm	{"bpm": 80-85}	0	1 s
Bed1-Ventilator	hosp/icu/bed1/vent/pressure	{"peak_press": 20-24}	1	1 s
Bed1-InfusionPump	hosp/icu/bed1/pump/drug_a	{"rate": 10, "status": "running"}	2	5 s
Bed1-Urometer	hosp/icu/bed1/fluid/urine	{"vol_ml": 250-255}	1	30 s
Bed1-TempProbe	hosp/icu/bed1/sens/temp	{"temp_c": 36-37}	1	60 s
Bed1-SmartBed	hosp/icu/bed1/furn/scale	{"weight_kg": 78}	0	300 s
Room1-EnvSensor	hosp/icu/room1/env/quality	{"co2_ppm": 410-430}	0	120 s

- **DoS Flood:** An attack designed to overwhelm the broker's bandwidth and processing capacity. The attack utilized 50 concurrent clients, flooding the broker with approximately 15 MB of payload data per client.
- **Slow DoS [72]:** A connection exhaustion attack aimed at saturating the broker's available file descriptors. This involved establishing numerous simultaneous MQTT connections, holding them open to prevent legitimate devices from connecting, and saturating the broker's resources.
- **Crafted Packets:** Injection of malformed or non-compliant MQTT packets. This vector tests the broker's parser robustness by sending manipulated packet headers and payloads, attempting to trigger crashes or unexpected behavior in the broker.

As previously discussed, the generation strategy employed in this study relies on an experimental setup featuring a Mosquitto [108] broker and synthetic IoMT devices generated via IoTflock [73]. Before deploying mqttsa [102], a sanity check is performed to verify that all virtual devices managed by IoTflock are successfully connected to the broker. Once this initialization phase is complete, the attack scenarios are executed: while IoTflock continuously generates benign background traffic, mqttsa is periodically launched to simulate the aforementioned attacks. Crucially, both benign and malicious clients operate concurrently against the exact same Mosquitto broker within the same test network. mqttsa[102] provides other types of attacks, but they do not apply due to the configuration of IoTflock.

The final composition of the datasets is presented in Table 3.

5.3 Feature Engineering & preprocessing

To ensure the model learns actual attack patterns rather than memorizing specific sessions, the resulting feature set $\mathcal{F}$ is selected with a focus on header fields that define the intent of the communication. Table 4 details the selected features. In the Feature preprocessing stage fields a translation

Naturally, the default SurIoT packet selection mode $\mathcal{M}$ is volumetric mode; on the other hand, for injection Attacks (Crafted Packets), the non-volumetric mode is selected.

5.4 ML/DL Classification & Suricata Testing Model Selection for Comparative Analysis

In this phase, we conduct the core comparative analysis of the experimental workflow. Specifically, the pre-processed network features are utilized to train and evaluate a diverse set of baseline Machine Learning (ML) and Deep Learn-

ing (DL) models. Concurrently, the original raw network traffic is processed directly by the Suricata engine, which is configured with the deterministic rules synthesized by the SurIoT approach. This dual-track execution is critical: it not only rigorously validates the semantic fidelity of the rule translation process but also allows us to benchmark the classification performance and computational efficiency of our interpretable approach against traditional, often opaque, data-driven pipelines.

- **Decision Tree (DT) [109]:** The core of the SurIoT approach, selected for its native interpretability and direct translatability into Boolean Suricata signatures.
- **Random Forest (RF) [110]:** An ensemble baseline used to evaluate the performance trade-off between complex multi-path models and the single-tree logic required for rule synthesis.
- **Gaussian Naive Bayes (NB) [111]:** A probabilistic benchmark used to assess whether tree-based non-linear splits provide a significant advantage over simple statistical feature independence.
- **Neural Network (NN) [112]:** A connectionist "black-box" model that serves as a performance ceiling, capturing complex dependencies to measure the gap between deep learning and interpretable rules.

5.4.1 Hyperparameter Optimization

The hyperparameter optimization of the SurIoT model was conducted through an exhaustive Grid Search, prioritizing the generalization capabilities of the synthesized rules. As outlined in Table 5, specific search spaces were defined tailored to the architectural characteristics of each evaluated algorithm.

5.5 Evaluation Metrics

- **Accuracy:** Represents the global ratio of correctly classified instances over the total number of samples. It serves as a baseline indicator of the model's general reliability:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

- **Precision:** Quantifies the fidelity of positive predictions. It measures the probability that a sample classified as an attack is indeed malicious, thereby reflecting the system's resilience against false alarms:

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

Table 3 Traffic distribution across the evaluated datasets

Dataset	Benign (0)	Malicious (1)	Total Packets
Craft	57,688 (97.37%)	1,558 (2.63%)	59,246
DoS Flood	319,980 (51.12%)	305,950 (48.88%)	625,930
Slow DoS	464,676 (46.68%)	530,764 (53.32%)	995,440
IoMT-Traffic-Data	2,265,164 (15.64%)	12,218,589 (84.36%)	14,483,753

Table 4 Chosen Network Features and Technical Definitions

Feature	Description
ip.proto	Protocol field in the IP header identifying the transport layer protocol (e.g., TCP).
ip.ttl	Time To Live (TTL) field indicating the maximum number of hops a packet is allowed to traverse.
tcp.len	Length of the TCP segment payload in bytes (excluding headers).
tcp.flags.syn	SYN : Synchronize sequence numbers; used to initiate a connection.
tcp.flags.ack	ACK : Acknowledgment field is significant; confirms receipt of data.
tcp.flags.fin	FIN : No more data from sender; used to terminate a connection.
tcp.flags.rst	RST : Reset the connection immediately.
tcp.flags.psh	PSH : Push function; instructs the receiver to push buffered data to the application.
tcp.flags.urg	URG : Urgent pointer field is significant.
tcp.flags.ece	ECE : ECN-Echo; indicates that the TCP peer is ECN capable.
tcp.flags.cwr	CWR : Congestion Window Reduced; indicates that the sender reduced its sending rate.

Table 5 Models and hyperparameter search spaces used in the grid search (5-fold stratified CV, scoring: F1)

Model	Hyperparameter	Search Space / Value
Decision Tree	max_depth	{3, 4, 5, 6, 7}
	min_samples_split	{5, 10, 100, 500, 1000}
	min_samples_leaf	{1, 2, 10, 100, 200}
	criterion	{gini, entropy}
Random Forest	n_estimators	{50, 100, 200}
	max_depth	{None, 10, 20}
	class_weight	balanced
Gaussian Naive Bayes	var_smoothing	1×10^{-9}
Neural Network (NN)	hidden_layer_sizes	(64, 32)
	activation	ReLU
	max_iter	500
Global Seed: 42		

- **Recall**: Also defined as the True Positive Rate (TPR), this metric assesses the model's capacity to detect all actual malicious instances. High recall is critical in intrusion detection to minimize the risk of missed attacks:

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

- **F1-Score**: The harmonic mean of Precision and Recall. This metric offers a balanced view of the classifier's performance, ensuring that neither false positives nor false

negatives disproportionately skew the evaluation:

$$F1-Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (4)$$

- **Inference Time (Latency)**: The wall-clock time required to classify a test set, expressed in seconds (s) and reported as the mean and standard deviation ($\mu \pm \sigma$) over $N = 3$ independent repetitions per PCAP to filter transient OS jitter. To ensure a fair comparison, timing boundaries are defined as follows:

- *ML models*: The timer covers the full deployment pipeline, computed as $T_{tot} = T_{ishark} + T_{preproc} +$

T_{predict} . This encompasses raw packet extraction via `tshark`, tabular pre-processing, and the final model prediction.

- *Suricata and Snort*: Measured by replaying the PCAP in single-threaded `runmode` with output plugins disabled, thus isolating the pure detection cost.

Low latency is a strict requirement for IoMT systems to operate in real-time without introducing network delays.

- **Throughput**: Defined as the number of packets processed per unit of time, expressed in packets per second (pps) and computed from the mean total processing time over the $N=3$ repetitions. It quantifies the system's capacity to handle high-volume traffic loads, such as those generated during DoS Flood attacks:

$$\text{Throughput} = \frac{N_{\text{packets}}}{T_{\text{total}}} \quad (5)$$

where N_{packets} is the total volume of traffic and T_{total} is the total processing duration including feature extraction and classification.

5.6 Results and Discussion

To provide a comprehensive and structured evaluation, the remainder of this section is articulated into three distinct parts. First, Section 5.6.1 details the classification metrics and the effectiveness of the generated rules, breaking down the approach behavior across isolated attack vectors as well as the aggregated, heterogeneous IoMT-Traffic-Data corpus. Subsequently, Section 5.6.2 assesses the practical viability of the proposed approach by evaluating its computational complexity, via the real-time inference throughput of the resulting IDS engines. Finally, Section 5.6.3 conducts a methodological and qualitative confrontation of our findings against recent literature, highlighting the differences in data preprocessing and the deployment advantages of our fully explainable approach.

The experimental results, summarized in Table 6 and Figures 4 - 7, validate the efficacy and reliability of the SurIoT approach. Table 6 provides a comprehensive overview of four key classification metrics across the evaluated datasets and detectors. The results are logically partitioned into two sections: the top tier reports on three synthetic datasets designed to stress-test individual attack vectors, while the bottom tier details the performance on the aggregated, IP-labelled IoMT-Traffic-Data corpus. This latter dataset closely mirrors the structural complexity of the corpus introduced by Areia et al. [100], serving as our primary reference scenario for state-of-the-art comparisons. Complementing these classification metrics, lastly Table 7 compares the overall processing times and real-time inference throughput across the different deployment engines.

5.6.1 Detection Performance: Scenario-Based Evaluation

Crafted Scenario

The Crafted scenario remains the most clear-cut case: the Decision Tree achieves a perfect F1 of 100.00%, supported by identical maximum scores in accuracy, precision, and recall. This is the by-product of the targeted preprocessing applied to non-volumetric attacks, which discards standard handshake packets and keeps only the data-carrying ones. In the resulting feature space, the malicious payloads exhibit drastically different length characteristics from the benign telemetry, which makes the classification task trivial and yields a zero false-positive rate. When comparing against the baseline models, the Random Forest perfectly matches this outcome. The Neural Network also remains highly competitive (F1 of 99.36%). Conversely, Naive Bayes struggles severely, showing a recall of just 6.73% and an F1 of 12.61%: this drastic drop indicates that its conditional independence assumption is highly inadequate for managing the structural dependencies of payload length features. More specifically, the Naive Bayes failure in this scenario can be attributed to its inability to model the joint distribution of correlated packet-level attributes: when multiple features carry redundant information about the same underlying payload structure, the independence assumption leads to a systematic underestimation of the posterior probability for the malicious class, causing the classifier to abstain from flagging the vast majority of attack packets. The Suricata engine reproduces this DT result exactly (Table 6).

DoS Flood Scenario

The DoS Flood scenario reaches an F1 of 99.77% and an overall accuracy of 99.78% on the DT, and is reproduced identically by Suricata. The volumetric nature of the attack creates a feature distribution that is well separated from the heterogeneous patterns of legitimate communications, and the DT carves precise boundaries on packet size and flag combinations: the near-perfect precision of 99.99% ensures that the system triggers virtually no false alarms, while the strong recall (99.55%) guarantees that only a sub-percent fraction of attack packets eludes detection. Across the ML baselines, performance is remarkably uniform. The Random Forest matches the DT identically ($F1 = 99.77\%$), and the Neural Network follows closely ($F1 = 99.75\%$). Even Naive Bayes performs adequately in this specific scenario ($F1 = 99.33\%$), confirming that volumetric flood attacks generate easily separable and independent feature distributions. This uniformity across structurally different classifiers is itself a noteworthy result: it suggests that the flood-induced signal in the feature space is so dominant that the specific inductive bias of the learner becomes largely irrelevant, and even the weakest model can exploit the separation with minimal accuracy loss. From a deployment perspective, this reinforces the rationale for choosing the simplest viable model, the Deci-

sion Tree, since it yields equivalent detection quality while enabling direct translation into deterministic Suricata rules without any loss of discriminative power.

Slow DoS Scenario

The Slow DoS scenario, by contrast, yields a comparatively lower F1 of 83.08% and an accuracy of 83.03% on the joined subset (DT and Suricata match digit-for-digit). The drop is intrinsic to the attack's operating logic, which exploits the very same TCP handshake states (SYN, SYN-ACK, FIN) used by legitimate connection setups. Because the per-packet feature representation has no temporal dimension, no flow duration, no inter-arrival statistics, the DT cannot contextualise the connection state, and an unavoidable overlap is observed at the handshake boundary. The metric most affected is precision (78.95%), while recall holds slightly better at 87.67%. This indicates a propensity for false positives (alert fatigue) rather than missed detections, which is a limitation of the underlying model, not of the SurIoT translation. Comparing this to the baselines, the Random Forest and Neural Network achieve a marginally higher F1 of 84.77% (precision 82.97%, recall 86.65%). While these more complex architectures can extract slightly better non-linear boundaries, they cannot fully overcome the lack of flow-level temporal context. Interestingly, Naive Bayes maximizes precision (100.00%) but completely collapses in recall (19.33%). This extreme behaviour of Naive Bayes deserves further comment: by assigning near-zero posterior probability to the malicious class for the vast majority of ambiguous handshake packets, the classifier effectively abstains from detection altogether, producing a trivially high precision at the cost of missing over 80% of attack traffic. Such a configuration would be operationally useless in any real deployment, as the fundamental purpose of an IDS is to maximise threat visibility rather than to minimise false alarms at the expense of near-total blindness. The IDS/IPS dichotomy mitigates the issue in practice: in passive (IDS) mode, the operator is alerted that an attack is in progress and the imperfect classification is no longer critical, whereas in active (IPS) mode, the rules would block ~87% of attack traffic and ~21% of benign traffic containing the attack while still generating actionable alerts. It is also worth noting that the performance ceiling observed across all four classifiers in this scenario constitutes indirect evidence that the bottleneck is representational rather than algorithmic: no amount of model sophistication can compensate for the absence of temporal flow features when the attack mimics the exact packet-level profile of legitimate TCP session establishment. Addressing this limitation would require augmenting the feature set with inter-arrival time statistics or connection-state tracking, which is left as a direction for future work.

IoMT-Traffic-Data Scenario

The aggregated IP-based scenario is qualitatively different from the previous three: a single Decision Tree is trained

on a heterogeneous corpus of seven PCAPs and over 14.4M packets where benign and malicious traffic are labelled at IP granularity. Note that Layer-2-only attacks (ARP Spoofing and CAM Table Overflow) are excluded from training and evaluation, as they carry no IP header and are therefore unreachable by the current feature set (cf. Section 5.2.1). Despite the structural diversity of the remaining attack types (Net Scan, MQTT Malaria, RUDY, Slow Loris, Slow Read, Apache Killer) the DT reaches an F1 of 99.85%, an accuracy of 99.73%, and a highly balanced precision (99.87%) and recall (99.83%) on the joined subset, and the translated Suricata rule base reproduces this score digit-for-digit (Table 6). This indicates that the rule generator does not lose discriminative power when the training distribution becomes more representative of a real network. A crucial observation emerges from the baseline evaluation: the structurally simple Decision Tree performs on par with, and even marginally outperforms, the Random Forest ($F1 = 99.83\%$) and exactly matches the Neural Network ($F1 = 99.85\%$). This confirms that the intrinsic complexity of the IoMT-Traffic-Data corpus can be effectively captured by a hierarchical set of orthogonal splits, rendering the DT an optimal, fully explainable surrogate model without any sacrifice in detection power. Once again, Naive Bayes lags significantly behind ($F1 = 69.38\%$), heavily penalised by the correlated nature of real-world network traffic. The fact that Naive Bayes degrades catastrophically on the aggregated corpus despite achieving adequate performance on the DoS Flood subset further illustrates how the conditional independence assumption becomes increasingly untenable as the number and diversity of attack types grow. In a heterogeneous traffic mix, inter-feature correlations multiply and interact in ways that a factored probabilistic model cannot capture, leading to a systematic miscalibration of class posteriors. By contrast, the Decision Tree's greedy, axis-aligned splitting strategy naturally adapts to the local geometry of each subregion of the feature space, handling correlated features through successive conditional partitions without requiring any explicit decorrelation or dimensionality reduction step.

5.6.2 Computational Complexity and Operational Analysis

The time taken to generate the rules is negligible, because this is a linear algorithm applied to a decision tree; the time across all datasets is less than a second. The inference-time comparison (Figure 4 - 7 and Table 7) is the most consequential finding of the operational evaluation. As described in Section 5.5, the ML pipeline inference time is decomposed into three sequential phases: (i) packet field extraction with *tshark* (T_{tshark}), (ii) NumPy preprocessing (T_{preproc}), and (iii) the *predict* call of the classifier (T_{predict}). All three phases are measured independently over $N=3$ repetitions; the total inference time and its standard deviation are

Table 6 Comparative classification performance (%) across all evaluated datasets and detectors. The double rule separates the synthetic single-attack scenarios (top) from the aggregated IoMT-Traffic-Data corpus (bottom). DT, RF, NB, and NN refer to the four ML models trained by SurIoT; Snort and Suricata refer to the engines driven by the rules translated from the best DT. Therefore, they have the same scores

Dataset	Detector	Accuracy	Precision	Recall	F1
Craft	Decision Tree	100.00	100.00	100.00	100.00
	Random Forest	100.00	100.00	100.00	100.00
	Naive Bayes	97.54	100.00	6.73	12.61
	Neural Network	99.97	98.73	100.00	99.36
	Suricata	100.00	100.00	100.00	100.00
DoS Flood	Snort	100.00	100.00	100.00	100.00
	Decision Tree	99.78	99.99	99.55	99.77
	Random Forest	99.78	99.99	99.55	99.77
	Naive Bayes	99.36	99.99	98.68	99.33
	Neural Network	99.76	99.97	99.53	99.75
Slow DoS	Suricata	99.78	99.99	99.55	99.77
	Snort	99.78	99.99	99.55	99.77
	Decision Tree	83.03	78.95	87.67	83.08
	Random Forest	85.20	82.97	86.65	84.77
	Naive Bayes	61.65	100.00	19.33	32.39
IoMT-Traffic-Data	Neural Network	85.20	82.97	86.65	84.77
	Suricata	83.03	78.95	87.67	83.08
	Snort	83.03	78.95	87.67	83.08
	Decision Tree	99.73	99.87	99.83	99.85
	Random Forest	99.71	99.88	99.78	99.83
	Naive Bayes	58.34	99.96	53.12	69.38
	Neural Network	99.73	99.86	99.83	99.85
	Suricata	99.73	99.87	99.83	99.85
	Snort	99.73	99.87	99.83	99.85

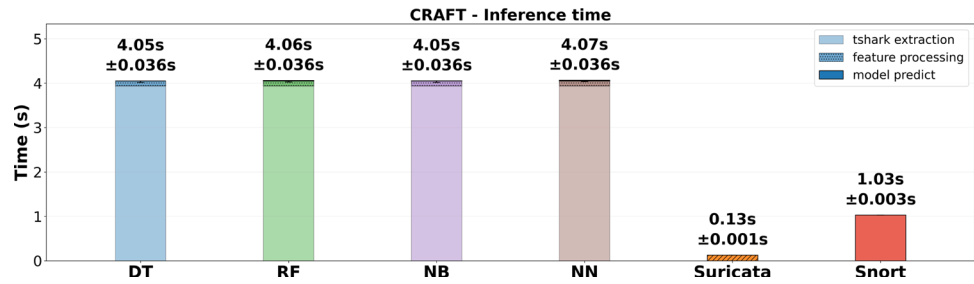
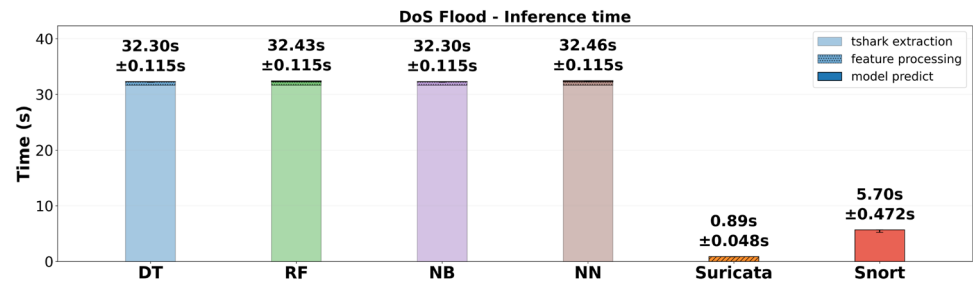
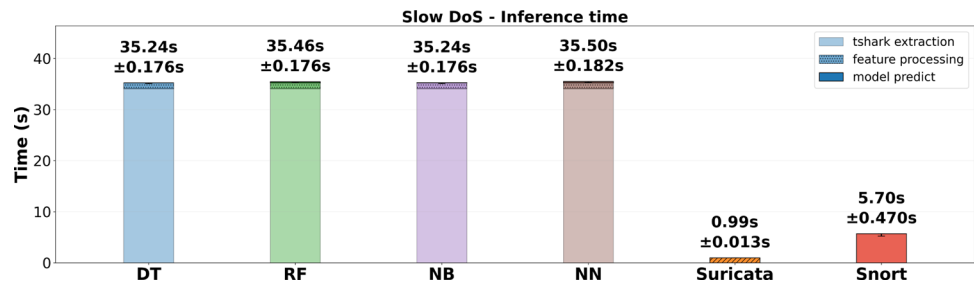
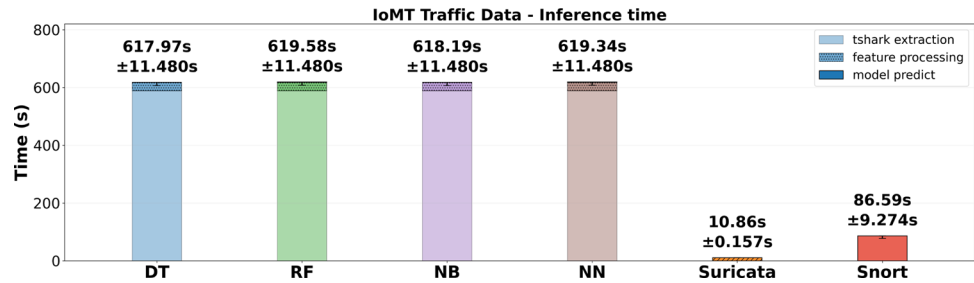
obtained by summation and quadrature propagation, respectively. Crucially, the `predict` call is negligible in every configuration, a few milliseconds at most, even for the Random Forest and the Neural Network on the largest corpus, and the entire ML cost is structurally absorbed by the upstream feature-extraction stage (`tshark` alone accounts for $> 97\%$ of the total time on all four datasets). Suricata and Snort are timed by replaying the same PCAP in single-threaded mode with all output plugins disabled, Fig. 5 repeated $N=3$ times per PCAP and aggregated as $\mu \pm \sigma$. Suricata is consistently faster than both the ML pipeline and Snort: in the Default scenarios Suricata processes Craft in 0.13 s versus 4.05 s of the DT pipeline ($\sim 31\times$), DoS Flood in 0.89 s versus 32.30 s ($\sim 36\times$) and Slow DoS in 0.99 s versus 35.24 s ($\sim 36\times$). On the IoMT-Traffic-Data corpus, Fig. 6 the gap becomes even more pronounced: 10.86 s for Suricata against ~ 618 s for the full ML pipeline ($\sim 57\times$). Snort is roughly one order of magnitude slower than Suricata, a known consequence of the different threading model, but still surpasses the ML pipeline by a factor of four to seven.

The implication for deployment is twofold. First, the rule generation phase is paid only once per model, while the operational advantage of the resulting Suricata signatures persists throughout the lifetime of the deployment. Second,

even when the underlying ML model is as light as a Decision Tree, the cost of feeding the classifier (`tshark` + preprocessing) is structurally unavoidable: more sophisticated detectors proposed in the literature [22, 27, 29] can only widen the gap, since the feature-extraction overhead is a lower bound common to all ML approaches. SurIoT therefore consistently dominates the ML alternative on every dataset evaluated in this work, with an additional benefit on the IoMT-Traffic-Data corpus, where the throughput gap grows by an order of magnitude.

5.6.3 Comparative Evaluation

A direct quantitative confrontation with the literature is possible on the IoMT-Traffic-Data side of the evaluation, since our aggregated IP-based corpus is derived from the same dataset introduced by Areia et al. [100] and subsequently used by Turgut et al. [22] (XBiDeep). However, before comparing numbers, the methodological differences that separate the three works must be stated clearly and explicitly, as they preclude a strictly controlled, apples-to-apples comparison:

Fig. 4 Inference time for the Craft dataset**Fig. 5** Inference time for the DoS Flood dataset**Fig. 6** Inference time for the Slow DoS dataset**Fig. 7** Inference time for the IoMT-Traffic-Data dataset**Table 7** Processing times (mean ± std over 3 repetitions per PCAP) and throughput. Throughput is computed against the post-processing packet count (CSV size)

Metric	Craft	DoS Flood	Slow DoS	IoMT-Traffic-Data
Processed packets	59 246	625 930	995 440	14 483 753
<i>Processing time (s)</i>				
ML pipeline (Total, DT)	4.05 ± 0.04	32.30 ± 0.12	35.24 ± 0.18	617.97 ± 11.48
Suricata (Total)	0.130 ± 0.001	0.894 ± 0.048	0.989 ± 0.013	10.86 ± 0.16
Snort (Total)	1.033 ± 0.003	5.698 ± 0.472	5.701 ± 0.470	86.59 ± 9.27
<i>Throughput (pps)</i>				
ML pipeline	14 614	19 380	28 254	23 437
Suricata	455 935	700 332	1 006 170	1 333 576
Snort	57 370	109 857	174 607	167 269

- (a) *Raw PCAP versus curated subset*. The most fundamental divergence concerns how each work consumes the underlying dataset. SurIoT processes the complete set of Layer-4-reachable PCAP captures as described in Section 5.2.1: no packet is discarded, no balancing is applied, and the entire traffic volume is fed into the pipeline. By contrast, Areia et al. [100] apply an extensive preprocessing pipeline to a curated subset of the data, comprising six steps: outlier removal, data mapping and cleaning, one-hot encoding of categorical variables, z-score normalisation, and unnecessary-column removal. Turgut et al. [22] similarly use a balanced sub-selection of 500,000 benign and 500,000 malicious samples (62,500 per attack type), from which they additionally remove a temporal feature before training. As a consequence, the three works train and evaluate on structurally different subsets of the same raw material, making a direct 1:1 mapping of results impossible.
- (b) *Binary classification is only a structural constraint of Suricata/Snort*. Areia et al. [100] report both binary and multi-class results; Turgut et al. [22] perform exclusively nine-class classification. SurIoT performs only binary classification (benign vs. malicious). This is not a design preference but a structural constraint: Suricata and Snort rules are intrinsically Boolean; a rule either fires (alert) or does not. Multi-class discrimination, i.e., assigning an individual attack subtype to each packet, is not currently supported by the rule-translation stage of the SurIoT pipeline and would require a fundamentally different rule representation. Multi-class extension is a natural direction for future work, but it falls outside the scope of the present system.
- (c) *Attack scope*. As detailed in Section 5.2.1, SurIoT explicitly excludes Layer-2-only attacks (ARP Spoofing and CAM Table Overflow) because they carry no IP header and are unreachable by the current packet-level feature extractor. These attacks may be represented in the reference works through flow-level aggregation or alternative feature sets, introducing a further asymmetry.
- (d) *Feature representation*. Areia et al. [100] evaluate both IP-based packet features and flow-level statistics; Turgut et al. [22] use 24 flow features as temporally ordered sequences for their recurrent architecture. SurIoT derives all features directly from raw packet fields via an automated `tshark` extraction pass, with no manual feature engineering, no flow aggregation, and no scaling.

Furthermore, it is conceptually crucial to distinguish SurIoT's evaluation context from recent generative rule-creation frameworks discussed in Section 3, such as RuleXploit [56] or Hex2Sign [55]. While those generative works evaluate the semantic similarity of their output rules to natural

language labels or exploit code (e.g., relying on NLP metrics like BERTscore), SurIoT evaluates direct enforcement accuracy on raw traffic. Attempting to compare traditional classification metrics against semantic translation metrics would introduce a profound epistemological mismatch.

Despite all the caveats above, the results reveal a set of observations that are robust to the methodological differences:

Competitive Accuracy on Unprocessed Data

SurIoT achieves detection accuracy that is fully competitive with, and in some cases exceeds, the results obtained by both reference works, even though it operates on completely unprocessed PCAP data with no feature engineering. The SurIoT Decision Tree obtains 99.85% F1 and 99.83% Recall on the IoMT-Traffic-Data corpus, matching or surpassing most classifiers reported by Areia et al. [100] including their Logistic Regression (F1=92.07%), Naive Bayes (92.94%), and SVM (92.08%) and closely approaching the best-performing Random Forest (F1=99.89%). The significant performance gap between our simple DT and classical distance-based baselines (like SVM and Logistic Regression) underscores a crucial methodological point: linear and distance-based models strictly depend on rigorous data scaling and feature normalisation to converge effectively. In contrast, the hierarchical splitting logic of our DT intrinsically handles heterogeneous, unscaled raw packet fields without requiring standardisation.

Similarly, SurIoT's Random Forest reaches F1=99.83%, within 0.12 pp of the Areia et al. RF, starting from raw packet data instead of a carefully pre-processed feature set. Against the deep BiGRU-BiLSTM of Turgut et al. [22] (F1=99.50% in multi-class), SurIoT's binary classifiers match or exceed the accuracy despite the structural simplicity of the approach. This specific comparison reveals that the temporal sequence modelling provided by recurrent deep network baselines, while theoretically powerful for tracking flow states, introduces a computational overhead that is largely unnecessary for robust binary discrimination when the model is exposed to a massive, un-sampled packet-level training set.

Three factors likely explain why competitive accuracy is achievable on raw, unprocessed data. First, SurIoT trains on the full set of available packets, which is substantially larger than the balanced sub-samples used by both reference works; more training examples reduce bias and variance even without feature engineering. Second, binary grouping (benign vs. all attacks) is an inherently easier classification problem than nine-class discrimination: conflating all attack types into a single malicious class removes inter-class confusion that challenges multi-class learners. Third, evaluating our internal ML baselines confirms that increasing architectural complexity yields heavily diminishing returns on this corpus. Since our own Neural Network and Random Forest perform almost identically to the Decision Tree (Table 6), it is evident

that the raw data distribution is sufficiently rich and separable to be fully captured by basic orthogonal splits, negating the need for high-dimensional, non-linear mappings. This is further supported by the work of Areia et al. [100], where they confirm through a benchmark the validity of tree-based approaches.

Deployment Advantage and Explainability

SurIoT offers a qualitatively different deployment model that neither of the reference works addresses. By translating the Decision Tree into native Suricata/Snort rules, the ML inference path is replaced entirely with a packet-processing engine requiring no Python runtime, no feature-extraction step, and no numerical library at deployment time. The Suricata engine processes the same IoMT-Traffic-Data traffic in 10.86 ± 0.16 s, versus ~ 618 s for the full ML pipeline ($\sim 57 \times$ faster). No comparable inference-latency figure is reported in [100] or [22], suggesting that operational deployment cost is not the primary concern of those works, yet it is arguably the most critical constraint in real IoMT environments where detection latency and hardware budget are both tightly limited.

Furthermore, each Suricata rule produced by SurIoT is a human-readable conjunction of packet-level conditions that can be audited, modified, and deployed without any ML infrastructure. This form of white-box explainability is absent in both reference works and aligns with increasing regulatory pressure towards interpretable AI in safety-critical healthcare domains. Unlike probabilistic hybrid architectures (such as the neuro-fuzzy systems evaluated by Einy et al. [75]), which may introduce unpredictable false positives (alert fatigue) under dynamic load, the Boolean nature of our translated rules guarantees a deterministic response. This mitigates the "trust gap" often associated with autonomous mitigation systems in medical environments. Moreover, by strictly avoiding the heavy tensor operations required by Deep Learning baselines, SurIoT practically answers the call for lightweight edge-security deployment—a challenge frequently highlighted but rarely solved in operational testbeds (e.g., Zachos et al. [74]).

In summary, while a direct quantitative comparison is not fully warranted due to the methodological divergences above, the evidence is consistent with the conclusion that SurIoT's results are coherent with the state of the art in detection accuracy and simultaneously offer a qualitatively superior deployment model in terms of operational efficiency, explainability, and infrastructure simplicity.

5.7 Rules & Explainability

A crucial advantage of using Decision Tree classifiers lies in their inherent interpretability, which facilitates the translation of abstract statistical boundaries into operational detection signatures. The listings below present a representative sub-

set of the Suricata rules generated by SurIoT for the four datasets under analysis. These rules were emitted directly by the algorithm from the best-performing tree (Table 6) and are selected here to illustrate how the approach isolates the most discriminative logic for each attack vector. Unlike the black-box nature of deep models, this extraction provides a white-box validation: the classification relies on meaningful network artefacts, TCP flag permutations, and payload size bands, TTL thresholds, rather than coincidental correlations.

The DoS Flood rules in Listing 3 clearly bind the decision to a combination of payload size (`dsize`) and TCP flag patterns. The algorithm has learnt that legitimate MQTT telemetry in the simulated ICU environment lives in a narrow size band, and packets that fall outside that band, or that carry aggressive flag combinations (e.g. A, USCPEF, indicating an ACK co-occurring with the rare combination of urgent/push/sync/cwr/ECE/FIN bits), are systematically tagged as malicious. The mixed-distribution leaves (e.g. [0.508, 0.492]) reveal the residual ambiguity at the handshake boundary that we already observed in the metric analysis.

Listing 3 Selected Suricata rules for DoS Flood detection.

```
alert tcp-pkt any any -> any any (msg:"decision_tree for
class 1 with distribution [0.0, 1.0]";
dsize:<=89; tcp.flags: 0, USCPEF; ttl:>=160;
flow: stateless; sid:1000002; rev:1;)
alert tcp-pkt any any -> any any (msg:"decision_tree for
class 0 with distribution [1.0, 0.0]";
dsize: 3-22; tcp.flags: A, USCFE; flow: stateless;
sid:1000005; rev:1;)
alert tcp-pkt any any -> any any (msg:"decision_tree for
class 1 with distribution [0.013,
0.986]"; dsize: 21-27; tcp.flags: A, USCPEF;
flow: stateless; sid:1000007; rev:1;)
```

The Slow DoS rules in Listing 4 reflect the more complex nature of the attack. The DT combines TTL thresholds with TCP flags and, again, exposes leaves with non-degenerate distributions ([0.609, 0.391], [0.850, 0.150]) precisely on those patterns that the attack shares with legitimate handshakes. This is the structural reason why the F1 stops at 86.4 on this dataset, and why the same residual percentage of false positives shows up in the Suricata replay: the limitation lives in the underlying statistical model, while SurIoT translates the tree losslessly.

Listing 4 Selected Suricata rules for Slow DoS detection.

```
alert tcp-pkt any any -> any any (msg:"decision_tree for
class 1 with distribution [0.0001,
0.9999]"; ttl:<=159; tcp.flags: 0, USCPEF; flow: stateless;
sid:1000001; rev:1;)
alert tcp-pkt any any -> any any (msg:"decision_tree for
class 0 with distribution [1.0, 0.0]";
ttl:<=159; tcp.flags: A, UCPE; dsize:>=34; flow: stateless;
sid:1000003; rev:1;)
alert tcp-pkt any any -> any any (msg:"decision_tree for
class 0 with distribution [0.609,
```

```
0.391]"; ttl:<=159; tcp.flags: AF, UCPE; flow:stateless;
sid:1000004; rev:1;)
alert tcp-pkt any any -> any any (msg:"decision_tree for
class 1 with distribution [0.0, 1.0]";
ttl:>=160; flow:stateless; sid:1000008; rev:1;)
```

The Crafted IoMT scenario (Listing 5) demonstrates the model's ability to learn protocol-compliance constraints strictly through packet dimensions. Very small packets ($dsize \leq 8$) correspond to standard MQTT control messages (PINGREQ, PUBACK) and are recognised as benign, intermediate sizes correspond to nominal telemetry, while packets larger than 388 bytes are systematically labelled as malicious payload-injection attempts.

Listing 5 Selected Suricata rules for Crafted IoMT detection.

```
alert tcp-pkt any any -> any any (msg:"decision_tree for
class 0 with distribution [1.0, 0.0]";
dsize:<=8; flow:stateless; sid:1000001; rev:1;)
alert tcp-pkt any any -> any any (msg:"decision_tree for
class 0 with distribution [1.0, 0.0]";
dsize: 54-388; flow:stateless; sid:1000005; rev:1;)
alert tcp-pkt any any -> any any (msg:"decision_tree for
class 1 with distribution [0.0, 1.0]";
dsize:>=388; flow:stateless; sid:1000006; rev:1;)
```

Finally, on the aggregated IoMT-Traffic-Data as shown in Table 6, the translated ruleset reaches an F1 of 99.85. The rules in Listing 6 expose a particularly clean decision logic: very short packets ($dsize \leq 5$, typical TCP zero-window probes used by the volumetric attacks) and large packets ($dsize \geq 149$) with no flag set (0, RFUCSAE) or with the PSH bit (P, RFUCSAE) are flagged as malicious, while the narrow band 5–7 bytes typical of benign MQTT keep-alives is preserved as a clean negative class. This illustrates the practical pay-off of the SurIoT pipeline: even when trained on a heterogeneous, real-world dataset, the resulting ruleset remains short, intelligible, and easy to audit by a human operator.

Listing 6 Selected Suricata rules for the aggregated IoMT-Traffic-Data corpus.

```
alert tcp-pkt any any -> any any (msg:"decision_tree for
class 1 with distribution [0.0, 1.0]"; dsize:<=5;
flow:stateless; sid:1000001; rev:1;)
alert tcp-pkt any any -> any any (msg:"decision_tree for
class 0 with distribution [0.986, 0.014]"; dsize: 5-7;
flow:stateless; sid:1000002; rev:1;)
alert tcp-pkt any any -> any any (msg:"decision_tree for
class 1 with distribution [0.080, 0.920]";
dsize:>=149; tcp.flags: 0, RFUCSAE; flow:stateless;
sid:1000005; rev:1;)
alert tcp-pkt any any -> any any (msg:"decision_tree for
class 1 with distribution [0.001, 0.999]";
dsize:>=149; tcp.flags: P, RFUCSAE; flow:stateless;
sid:1000006; rev:1;)
```

Overall, these examples confirm the effectiveness of SurIoT at producing explainable rule bases. The same generative procedure scales seamlessly from a ~59k-packet single-attack scenario to a ~14.4M-packet, seven-PCAP

aggregate, and in both regimes the rules emitted by the approach can be inspected, validated, and ingested directly by the NIDPS engine of choice (Suricata or Snort) without further manual intervention.

5.7.1 Operational Deployment and Optimization

As outlined in Section 4.6.1, this section details the optimization phase, where the raw rules generated by the model are refined to meet specific, user-defined operational requirements. Unlike the generic classification logic produced by the Decision Tree, this stage allows the operator to tailor the rules for an ad-hoc context.

Through this example, the practical application of these ad-hoc optimizations is observable:

- **From Passive to Active Defense:** The operator explicitly requested an intervention strategy. Consequently, the approach replaced the standard alert action with the drop directive, converting the rule into an active IPS mechanism that immediately discards malicious packets at the interface level.
- **Context-Specific Scoping:** Instead of analyzing all traffic generically, the rule is restricted to a specific operational perimeter. By injecting environmental variables like \$HOME_NET and \$MQTT_PORTS, the detection is strictly bound to the critical IoMT infrastructure, eliminating noise from unrelated network segments.
- **Operational Integration:** To fit the specific monitoring needs of the organization, the rule is enriched with standardized metadata (e.g., sid, classtype, priority). This ensures that the generated alerts are not just isolated signals but are semantically consistent with the user's security architecture.

These types of optimizations, when requested by the user, provide two practical advantages. First of all, the transition to active defense, being a native function of Suricata, greatly speeds up the system's response compared to approaches based on AI. In fact, AI models only manage the traffic classification phase but do not obviously contain the functions for active defense, which is left to the network manager or other systems. When an AI-based system is not contained within a complex architecture capable of blocking packets, it only allows IDS mode, thus generating alerts without the ability to respond to the attack. SurIoT, on the other hand, overcomes this limitation. The second practical advantage is management related to a specific security architecture. Context-specific scoping allows rules to be adjusted so that they only run on the network segments of interest, avoiding a general overload of the network with consequent latency. Furthermore, operational integration allows for full incorpo-

ration into pre-existing security architectures, thus requiring no adaptations or major changes (as would be the case for models based on AI).

5.8 Robustness Considerations and Limitations

A concern for any signature-based detection system is its robustness against adversarial manipulation, where an attacker aware of the deployed rules deliberately crafts traffic to evade detection [113, 114]. This section discusses SurIoT's resilience to such strategies from a theoretical perspective.

The stateless features used by SurIoT are not easy to manipulate, because they are features tightly bound to protocol constraints. First, TCP flags are constrained by protocol correctness: changing flag combinations arbitrarily would break the TCP state machine, which would itself be caught by Suricata's built-in protocol anomaly detection [115]. Furthermore, the TTL field depends in part on the network path between attacker and target, so the attacker has limited control over its value [116]. Second, The `dsize` feature is easier to manipulate; for example, an attacker could fragment payloads to stay below a learned threshold [117]. However, this comes at a cost: a DoS Flood with smaller packets needs a higher packet rate to remain effective, which may expose other detectable patterns [118]. Similarly, a Crafted Packet attack that splits its malformed payload across fragments risks becoming ineffective against the target broker's parser [119].

More broadly, evasion is not a limitation specific to SurIoT; it affects all signature-based NIDPS equally [120]. Manually written Suricata rules are just as vulnerable when attackers reverse-engineer the deployed signatures. The key difference lies in response time: updating rules manually requires expert analysis and can take hours or days, while SurIoT can regenerate a new ruleset automatically by re-training on a PCAP that includes the evasive traffic. This shortens the attacker's window of opportunity and turns rule maintenance from a slow, manual process into a fast, semi-automated one. Consequently, by combining the inherent difficulty of manipulating structural protocol features with the ability to rapidly adapt to new threat patterns, SurIoT demonstrates a strong overall resilience against adversarial evasion tactics.

6 Conclusion and Future Work

This paper presented SurIoT, an approach designed to automate the generation of explainable Network Intrusion Detection and Prevention System (NIDPS) rules, specifically targeting the MQTT protocol in IoT and IoMT environments. By leveraging Decision Trees to extract detection logic from stateless network features, SurIoT bridges the gap between

machine learning accuracy and the high-speed, deterministic execution of rule-based engines like Suricata and Snort. Extensive experimental evaluations on both synthetic and real-world benchmark datasets (such as the IoMT-Traffic-Data corpus) confirm the viability of this approach. The rule translation process proved to be lossless, perfectly maintaining the discriminative power of the source models across all four evaluated scenarios. On the Crafted and DoS Flood datasets, both the Decision Tree and the translated Suricata rules achieved F1-scores of 100.00% and 99.77%, respectively, with the DoS Flood scenario further exhibiting a precision of 99.99% and a recall of 99.55%. On the aggregated IoMT-Traffic-Data corpus comprising over 14.4 million packets from seven heterogeneous PCAPs spanning six distinct attack types, the Decision Tree reached an F1 of 99.85%, an accuracy of 99.73%, and a balanced precision-recall pair of 99.87% and 99.83%, with the Suricata engine reproducing every digit without loss. Notably, the structurally simple Decision Tree matched or marginally outperformed both the Random Forest ($F1 = 99.83\%$) and the Neural Network ($F1 = 99.85\%$), confirming that complex, opaque architectures yield no meaningful detection advantage on this corpus. The only scenario where performance was substantially lower Slow DoS, with an F1 of 83.08% was shown to be a representational limitation intrinsic to stateless packet-level features rather than a deficiency of the model or the translation process, as all four classifiers converged to a similar performance ceiling. From a deployment perspective, SurIoT demonstrated a massive operational advantage: running the generated signatures in Suricata yielded throughput gains ranging from $31\times$ on the Craft dataset to $57\times$ on the IoMT-Traffic-Data corpus compared to the full ML pipeline. In absolute terms, Suricata processed 14.4 million packets in 10.86 ± 0.16 s (approximately 1.33 million packets per second), whereas the equivalent Python-based ML pipeline required 617.97 ± 11.48 s for the same workload. Even Snort, despite being roughly one order of magnitude slower than Suricata, still outperformed the ML pipeline by a factor of four to seven across all datasets. This effectively bypasses the computational bottleneck of real-time feature extraction, proving that the rules are computationally lightweight and suitable for resource-constrained e-health architectures. In conclusion, SurIoT provides a practical, transparent, and deployment-ready foundation for securing critical medical networks. Future research will focus on expanding the synthesis logic to support multi-class detection and exploring methods to integrate stateful protocol inspection without sacrificing reliability. Furthermore, we aim to investigate the integration of Generative AI (GenAI) and Large Language Models (LLMs) as a complementary future enhancement. Rather than replacing the deterministic core logic, which could introduce risks of hallucinations and false positives in critical settings [121, 122], GenAI modules could be

leveraged as an interpretability layer. This layer would automatically translate complex technical rule sets into clear, natural language reports for healthcare administrators [123], successfully balancing automated AI efficiency with transparent monitoring.

Author Contributions All authors contributed equally to the conception, design, execution, and writing of this work

Funding Open access funding provided by Università di Foggia within the CRUI-CARE Agreement. This research was co-funded by the Italian Complementary National Plan PNC-I.1 “Research initiatives for innovative technologies and pathways in the health and welfare sector” D.D. 931 of 06/06/2022, “DARE - DigitAI lifelong pRevEntion” initiative, code PNC0000002, CUP: B53C22006450001.

Data Availability Data will be made available on request

Materials Availability Not applicable.

Code Availability Code will be made available on request.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethics approval and consent to participate Not applicable.

Consent for publication Not applicable.

Use of Generative AI The authors declare that no generative AI was used in the writing of the paper.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. AbuAlghanam, O., Almobaideen, W., Othman, A.A., Lafi, H.Z.: Towards secure healthcare iot: a comprehensive survey on behavior-based authentication. *Pervasive Mob. Comput.* 102185 (2026)
2. Al Khatib, I., Shamayleh, A., Ndiaye, M.: Healthcare and the internet of medical things: applications, trends, key challenges, and proposed resolutions. In: *Informatics*, p. 47 (2024). MDPI
3. Osama, M., Ateya, A.A., Sayed, M.S., Hammad, M., Plawiak, P., El-Latif, A., Elsayed, A.A., R.A.: Internet of medical things and healthcare 4.0: Trends, requirements, challenges, and research directions. *Sensors* 23(17), 7435 (2023)
4. Ashfaq, Z., Rafay, A., Mumtaz, R., Zaidi, S.M.H., Saleem, H., Zaidi, S.A.R., Mumtaz, S., Haque, A.: A review of enabling technologies for internet of medical things (iomt) ecosystem. *Ain Shams Eng. J.* 13(4), 101660 (2022)
5. Kadhim, K.T., Alsahlany, A.M., Wadi, S.M., Kadhumi, H.T.: An overview of patient's health status monitoring system based on internet of things (iot). *Wireless Pers. Commun.* 114(3), 2235–2262 (2020)
6. Sadhu, P.K., Yanambaka, V.P., Abdelgawad, A., Yelamarthi, K.: Prospect of internet of medical things: a review on security requirements and solutions. *Sensors* 22(15), 5517 (2022)
7. Putra, K.T., Arrayyan, A.Z., Hayati, N., Damarjati, C., Bakar, A., Chen, H.-C., et al.: A review on the application of internet of medical things in wearable personal health monitoring: a cloud-edge artificial intelligence approach. *IEEE Access* 12, 21437–21452 (2024)
8. Cawthra, J.L., Ekstrom, M.R., Lusty, L.N., Sexton, J.T., Sweetnam, J.E.: Data integrity: detecting and responding to ransomware and other destructive events. Technical Report 1800-26, National Institute of Standards and Technology (NIST) (2020). <https://doi.org/10.6028/NIST.SP.1800-26>
9. Boutemour, J., Lella, I., Bakatsis, I., Chatzichristos, G., Foley, K., Leskinen, J., Otcenasek, J., Ziolek, D., European Union Agency for Cybersecurity (ENISA), EEAS StratCom: Enisa threat landscape 2025. Enisa report, European Union Agency for Cybersecurity (ENISA) (2025). <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2025>
10. Banks, A., Briggs, E., Borgendale, K., Gupta, R.: MQTT Version 5.0. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (2019)
11. Alshammari, H.H.: The internet of things healthcare monitoring system based on mqtt protocol. *Alex. Eng. J.* 69, 275–287 (2023)
12. Shelby, Z., Hartke, K., Bormann, C.: The Constrained Application Protocol (CoAP) - RFC 7252, Internet Engineering Task Force (IETF) (2014)
13. Alobaywi, B., Almutairi, M.G., Sheldon, F.T.: Performance trade-offs in multi-tenant iot-cloud security: a systematic review of emerging technologies. *IoT J.* 7(1), 21 (2026)
14. Dentamaro, V., Convertini, V.N., Galantucci, S., Giglio, P., Palmisano, T., Pirlo, G.: Ensemble consensus: an unsupervised algorithm for anomaly detection in network security data, pp. 309–318. *ITASEC* (2021)
15. Wang, Z., Li, J., Yang, S., Luo, X., Li, D., Mahmoodi, S.: A lightweight iot intrusion detection model based on improved bert-of-theseus. *Expert Syst. Appl.* 238, 122045 (2024)
16. Wang, Z., Chen, H., Yang, S., Luo, X., Li, D., Wang, J.: A lightweight intrusion detection method for iot based on deep learning and dynamic quantization. *PeerJ Comput. Sci.* 9, 1569 (2023)
17. Nie, F., Liu, W., Liu, G., Gao, B.: M2vt-ids: a multi-task multi-view learning architecture for designing iot intrusion detection system. *Int. Things* 25, 101102 (2024)
18. Musthafa, M.B., Huda, S., Kodera, Y., Ali, M.A., Araki, S., Mwaura, J., Nogami, Y.: Optimizing iot intrusion detection using balanced class distribution, feature selection, and ensemble machine learning techniques. *Sensors* 24(13), 4293 (2024)
19. Mirlashari, M., Rizvi, S.A.M.: Enhancing iot intrusion detection system with modified e-graphsage: a graph neural network approach. *Int. J. Inf. Technol.* 16(4), 2705–2713 (2024)
20. Elnakib, O., Shaaban, E., Mahmoud, M., Emara, K.: Eidm: deep learning model for iot intrusion detection systems. *J. Supercomputing* 79(12), 13241–13261 (2023)
21. Bouke, M.A., El Atigh, H., Abdullah, A.: Towards robust and efficient intrusion detection in iomt: a deep learning approach addressing data leakage and enhancing model generalizability. *Multimed. Tools Appl.* 84, 1–20 (2024)

22. Turgut, Z., Başarslan, M.S.: Xbideep: a novel explainable artificial intelligence based intrusion detection system for internet of medical things environment. *Int. Things* **33**, 101675 (2025)
23. Hosain, Y., Çakmak, M.: Xai-xgboost: an innovative explainable intrusion detection approach for securing internet of medical things systems. *Sci. Rep.* **15**(1), 22278 (2025)
24. Chaganti, R., Mourade, A., Ravi, V., Vemprala, N., Dua, A., Bhushan, B.: A particle swarm optimization and deep learning approach for intrusion detection system in internet of medical things. *Sustainability* **14**(19), 12828 (2022)
25. Hady, A.A., Ghubaish, A., Salman, T., Unal, D., Jain, R.: Intrusion detection system for healthcare systems using medical and network data: a comparison study. *IEEE Access* **8**, 106576–106584 (2020)
26. Kilincer, I.F., Ertam, F., Sengur, A., Tan, R.-S., Acharya, U.R.: Automated detection of cybersecurity attacks in healthcare systems with recursive feature elimination and multilayer perceptron optimization. *Biocybern. Biomed. Eng.* **43**(1), 30–41 (2023)
27. Mohammadi, A., Ghahramani, H., Asghari, S.A., Aminian, M.: Securing healthcare with deep learning: a cnn-based model for medical iot threat detection. In: 2024 19th Iranian Conference on Intelligent Systems (ICIS), pp. 168–173. IEEE (2024)
28. Ali, M., Saleem, Y., Hina, S., Shah, G.A.: Ddosvit: Iot ddos attack detection for fortifying firmware over-the-air (ota) updates using vision transformer. *Int. Things* **30**, 101527 (2025)
29. Akar, G., Sahnoud, S., Onat, M., Cavusoglu, Ü., Malondo, E.: L2d2: a novel lstm model for multi-class intrusion detection systems in the era of iomt. *IEEE Access* (2025)
30. Hossain, M.I., Ghosh, D., Islam, M.K., Auyon, M.O.S., Al Masud, A.: Cybersecurity in rural hospitals: reducing attack risks and financial losses. *J. Knowl. Manag. Pract.* **26**(1) (2026)
31. Jalali, M.S., Kaiser, J.P.: Cybersecurity in hospitals: a systematic, organizational perspective. *J. Med. Int. Res.* **20**(5), 10059 (2018)
32. Sgueglia, A., Di Sorbo, A., Visaggio, C.A., Canfora, G.: A systematic literature review of iot time series anomaly detection solutions. *Futur. Gener. Comput. Syst.* **134**, 170–186 (2022)
33. Naghib, A., Gharehchopogh, F.S., Zamanifar, A.: A comprehensive and systematic literature review on intrusion detection systems in the internet of medical things: current status, challenges, and opportunities. *Artif. Intell. Rev.* **58**(4), 114 (2025)
34. Suricata. <https://suricata.io/>
35. Pinto, R.P., Silva, B.M., Inácio, P.R.: Federated learning for anomaly detection on internet of medical things: a survey. *Int. Things*, 101677 (2025)
36. Alharthi, A., Alaryani, M., Kaddoura, S.: A comparative study of machine learning and deep learning models in binary and multi-class classification for intrusion detection systems. *Array*, 100406 (2025)
37. Gupta, K., Sharma, D.K., Gupta, K.D., Kumar, A.: A tree classifier based network intrusion detection model for internet of medical things. *Comput. Electr. Eng.* **102**, 108158 (2022)
38. Houda, E., Siham, A.: Ai in cybersecurity: A systematic survey of ml and dl techniques for enhanced intrusion detection. In: International Conference on Connected Objects and Artificial Intelligence, pp. 823–828. Springer (2025)
39. Pavan, S., Suhas, M., Yogesh, B., Surendra Babu, K., Thirumala Akash, K.: Intrusion detection landscape: Exploring progress and confronting challenges in security advances. In: 2024 International Conference on Integrated Circuits and Communication Systems (ICICACS), pp. 1–8. IEEE (2024)
40. Schrötter, M., Niemann, A., Schnor, B.: A comparison of neural-network-based intrusion detection against signature-based detection in iot networks. *Information* **15**(3), 164 (2024)
41. Nedungadi, N., Subran, A.K., Sankaran, S.: Comparative analysis of intrusion detection systems for internet of things. In: 2023 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS), pp. 1–6. IEEE (2023)
42. Nageswari, S., Kumar, B.A.: A comprehensive analysis of cyberattacks in internet of things. In: 2025 Third International Conference on Augmented Intelligence and Sustainable Systems (ICAIS), pp. 816–828. IEEE (2025)
43. Pragathi, B., Rao, K.S., Ramu, P., Rao, S.N.: Iot security assisted by machine learning: Unresolved problems and difficulties in the face of advanced persistent threats. In: AIP Conference Proceedings, vol. 3162, p. 020111 (2025). AIP Publishing LLC
44. Chen, Z., Liu, J., Shen, Y., Simsek, M., Kantarci, B., Mouftah, H.T., Djukic, P.: Machine learning-enabled iot security: open issues and challenges under advanced persistent threats. *ACM Comput. Surv.* **55**(5), 1–37 (2022)
45. Nawaal, B., Haider, U., Khan, I.U., Fayaz, M.: Signature-based intrusion detection system for iot, pp. 141–158. (2024)
46. Chiona, U.L., Kumar, R., Aggarwal, G.: A comprehensive survey on intrusion detection systems in iot environment. In: 2024 International Conference on Emerging Technologies and Innovation for Sustainability (EmergIN), pp. 122–127. IEEE (2024)
47. Husnain, M., Hayat, K., Cambiaso, E., Fayyaz, U.U., Mongelli, M., Akram, H., Ghazanfar Abbas, S., Shah, G.A.: Preventing mqtt vulnerabilities using iot-enabled intrusion detection system. *Sensors* **22**(2), 567 (2022)
48. Alcantara, L., Padilha, G., Abreu, R., d' Amorim, M.: Syrius: synthesis of rules for intrusion detectors. *IEEE Trans. Reliab.* **71**(1), 370–381 (2021)
49. Jacobs, R.: Tshark. <https://tshark.dev/>
50. Francis, G.T., Souri, A., İnanc, N.: A hybrid intrusion detection approach based on message queuing telemetry transport (mqtt) protocol in industrial internet of things. *Trans. Emerg. Telecommun. Technol.* **35**(9), 5030 (2024)
51. Bendele, B., Akopian, D.: A study of iot mqtt control packet behavior and its effect on communication delays. *Electronic Imaging* **29**, 120–129 (2017)
52. Aversano, L., Bernardi, M.L., Cimitile, M., Montano, D., Pecori, R., Veltri, L.: Explainable anomaly detection of synthetic medical iot traffic using machine learning. *SN Comput. Sci.* **5**(5), 488 (2024)
53. Georgiades, M., Hussain, F.: An explainable ai approach for interpretable cross-layer intrusion detection in internet of medical things. *Electronics* **14**(16), 3218 (2025)
54. Memon, S.A., Wiil, U.K., Shaikh, M.: Explainable intrusion detection for internet of medical things. In: 15th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management, pp. 40–51. SCITEPRESS Digital Library (2023)
55. Balasubramanian, P., Ali, T., Salmani, M., KhoshKholgh, D., Kostakos, P.: Hex2sign: automatic ids signature generation from hexadecimal data using llms. In: 2024 IEEE International Conference on Big Data (BigData), pp. 4524–4532 (2024). IEEE
56. Papoutsis, A., Dimitriadis, A., Koritsas, I., Kavallieros, D., Tsikrika, T., Vrochidis, S., Kompatsiaris, I.: Rulexploit: a framework for generating suricata rules from exploits using generative ai. In: 2025 IEEE International Conference on Cyber Security and Resilience (CSR), pp. 267–274. IEEE (2025)
57. Coscia, A., Dentamaro, V., Galantucci, S., Maci, A., Pirlo, G.: Automatic decision tree-based nids ruleset generation for dos/ddos attacks. *J. Inf. Secur. Appl.* **82**, 103736 (2024)
58. MQTT - The Standard for IoT Messaging [mqtt.org. https://mqtt.org/](https://mqtt.org/)
59. Waleed, A., Jamali, A.F., Masood, A.: Which open-source ids? snort, suricata or zeek. *Comput. Netw.* **213**, 109116 (2022)
60. Snort <https://www.snort.org/>
61. Zeek <https://zeek.org/>

62. Sarhan, M., Layeghy, S., Gallagher, M., Portmann, M.: From zero-shot machine learning to zero-day attack detection, (2021). [arXiv:2109.14868](https://arxiv.org/abs/2109.14868) arXiv preprint
63. Gupta, A., Sharma, L.S.: Performance evaluation of snort and suricata intrusion detection systems on ubuntu server, pp. 811–821. (2019)
64. Gosai, B.: Signature-based IDS and the Burden of Manual Rule Updates. https://www.theseus.fi/bitstream/handle/10024/910447/Gosai_Bhrugeshbharti.pdf
65. Batool, S., Aslam, M., Akpokodje, E., Jilani, S.F.: A comprehensive review of ddos detection and mitigation in sdn environments: machine learning, deep learning, and federated learning perspectives. *Electronics* **14**(21), 4222 (2025)
66. Masum, M., Shahriar, H.: A transfer learning with deep neural network approach for network intrusion detection. *Int. J. Intel. Comput. Res.* **12**(1), (2021)
67. Thirimanne, S.P., Jayawardana, L., Yasakethu, L., Liyanaarachchi, P., Hewage, C.: Deep neural network based real-time intrusion detection system. *SN Comput. Sci.* **3**(2), 145 (2022)
68. Dahal, A., Bajgai, P., Rahimi, N.: Analysis of zero day attack detection using mlp and xai. In: *World Congress in Computer Science, Computer Engineering & Applied Computing*, pp. 57–67 (2024). Springer
69. Suricata Documentation. <https://docs.suricata.io>
70. 8.50. Differences From Snort; Suricata 9.0.0-dev documentation docs.suricata.io. <https://docs.suricata.io/en/latest/rules/differences-from-snort.html#don-t-cross-the-streams>
71. 8.33. MQTT Keywords ; Suricata 9.0.0-dev documentation docs.suricata.io. <https://docs.suricata.io/en/latest/rules/mqtt-keywords.html#mqtt-keywords>
72. Vaccari, I., Aiello, M., Cambiaso, E.: Slowite, a novel denial of service attack affecting mqtt. *Sensors* (2020). <https://doi.org/10.3390/s20102932>
73. Ghazanfar, S., Hussain, F., Rehman, A.U., Fayyaz, U.U., Shahzad, F., Shah, G.A.: Iot-flock: an open-source framework for iot traffic generation. In: *2020 International Conference on Emerging Trends in Smart Technologies (ICETST)*, pp. 1–6. IEEE (2020)
74. Zachos, G., Mantas, G., Essop, I., Porfyraakis, K., Bastos, J.M.C., Rodriguez, J.: An iot/iomt security testbed for anomaly-based intrusion detection systems. In: *2023 IFIP Networking Conference (IFIP Networking)*, pp. 1–6. IEEE (2023)
75. Einy, S., Oz, C., Navaei, Y.D.: The anomaly-and signature-based ids for network security using hybrid inference systems. *Math. Probl. Eng.* **2021**(1), 6639714 (2021)
76. Alcantara, L., Padilha, G., Abreu, R., d'Amorim, M.: Syrius: synthesis of rules for intrusion detectors. *IEEE Trans. Reliab.* **71**(1), 370–381 (2021)
77. Zeghida, H., Boulaiche, M., Ramdane, C.: Xmid-mqtt: explaining machine learning-based intrusion detection system for mqtt protocol in iot environment. *Int. J. Inf. Secur.* (2025). <https://doi.org/10.1007/s10207-025-01036-w>
78. Georgiades, M., Hussain, F.: An explainable ai approach for interpretable cross-layer intrusion detection in internet of medical things. *Electronics* **14**(16), 3218 (2025). <https://doi.org/10.3390/electronics14163218>
79. Moreno, M., Iturbe, M., Sáez-de-Cámara, X., Urbieto, A.: Leveraging llms for automated ids rule generation: A novel methodology for securing industrial environments. Preprint/Working Paper (2025)
80. Li, J., et al.: Gridai: generating and repairing intrusion detection rules via collaboration among multiple llm-based agents. *arXiv preprint arXiv:2510.13257* (2025) <https://doi.org/10.48550/arXiv.2510.13257>
81. Boukebous, A.A.E., Fettache, M.I., Bendiab, G., Shialeles, S.: A comparative analysis of snort 3 and suricata. In: *2023 IEEE IAS Global Conference on Emerging Technologies (GlobConET)*, pp. 1–6. IEEE (2023)
82. Fadhilah, D., Marzuki, M.I.: Performance analysis of ids snort and ids suricata with many-core processor in virtual machines against dos/ddos attacks. In: *2020 2nd International Conference on Broadband Communications, Wireless Sensors and Powering (BCWSP)*, pp. 157–162. IEEE (2020)
83. Dadkhah, S., Neto, E.C.P., Ferreira, R., Molokwu, R.C., Sadeghi, S., Ghorbani, A.A.: Ciciomt 2024: a benchmark dataset for multi-protocol security assessment in iomt. *Int. Things* **28**, 101351 (2024)
84. Kaur, P., Kumar, R., Kumar, M.: A healthcare monitoring system using random forest and internet of things (iot). *Multimed. Tools Appl.* **78**(14), 19905–19916 (2019)
85. Kulshrestha, P., Vijay Kumar, T.: Machine learning based intrusion detection system for iomt. *Int. J. Syst. Assur. Eng. Manag.* **15**(5), 1802–1814 (2024)
86. Mahbooba, B., Timilsina, M., Sahal, R., Serrano, M.: Explainable artificial intelligence (xai) to enhance trust management in intrusion detection systems using decision tree model. *Complexity* **2021**(1), 6634811 (2021)
87. Patil, S., Varadarajan, V., Mazhar, S.M., Sahibzada, A., Ahmed, N., Sinha, O., Kumar, S., Shaw, K., Kotecha, K.: Explainable artificial intelligence for intrusion detection system. *Electronics* **11**(19), 3079 (2022)
88. Wali, S., Farrukh, Y.A., Khan, I.: Explainable ai and random forest based reliable intrusion detection system. *Computers & Security* **157**, 104542 (2025)
89. Mareedu, A.: Machine learning applications in intrusion detection: a comprehensive review. *Int. J. Multidiscip. Sci. Manag.* **1**(4), (2024)
90. Shah, S.A.R., Issac, B.: Performance comparison of intrusion detection systems and application of machine learning to snort system. *Futur. Gener. Comput. Syst.* **80**, 157–170 (2018)
91. Open Information Security Foundation: Rule Types and Categorization. <https://docs.suricata.io/en/latest/rules/rule-types.html> (2025)
92. Open Information Security Foundation: Inspection of Raw Stream Data. https://docs.suricata.io/en/latest/devguide/internals/engines/stream/inspection_raw_data.html (2025)
93. Open Information Security Foundation: Header Keywords. <https://docs.suricata.io/en/latest/rules/header-keywords.html> (2025)
94. tcp, tcp-pkt, tcp-stream in Suricata rule hawu.me. <https://www.hawu.me/ops/4843>
95. Ahmed, M., Byreddy, S., Nutakki, A., Sikos, L.F., Haskell-Dowland, P.: Ecu-ioht: a dataset for analyzing cyberattacks in internet of health things. *Ad Hoc Netw.* **122**, 102621 (2021)
96. Alsaman, D.: A comparative study of anomaly detection techniques for iot security using amot (adaptive machine learning for iot threats). *IEEE Access* pp. 1–1 (2024) <https://doi.org/10.1109/ACCESS.2024.3359033>
97. 8.8. Integer Keywords ; Suricata 9.0.0-dev documentation docs.suricata.io. <https://docs.suricata.io/en/latest/rules/integer-keywords.html>
98. 8.3. IP Keywords ; Suricata 9.0.0-dev documentation docs.suricata.io. <https://docs.suricata.io/en/latest/rules/header-keywords.html#tcp-flags>
99. 8.53. Differences From Snort. <https://docs.suricata.io/en/latest/rules/differences-from-snort.html>
100. Areia, J., Bispo, I.A., Santos, L., Costa, R.L.D.C.: Iomt-trafficdata: dataset and tools for benchmarking intrusion detection in internet of medical things. *IEEE Access* **12**, 115370–115385 (2024)
101. GitHub - ivoafonsobispo/IoMT-AI-IDS: Machine Learning-Based Intrusion Detection System (IDS) for the Internet

- of Medical Things (IoMT) github.com. <https://github.com/ivoafonsobispo/IoMT-AI-IDS>
102. Palmieri, A., Prem, P., Ranise, S., Morelli, U., Ahmad, T.: Mqttsa: a tool for automatically assisting the secure deployments of mqtt brokers. *IEEE World Congress on Services (SERVICES)* **2642**, 47–53 (2019)
 103. Shekhan, S.: slowhttpstest: a tool for simulating slow http attacks. <https://github.com/shekhan/slowhttpstest> (2024)
 104. Rodríguez, L.: mqtt-malaria: a tool for testing and exploiting mqtt brokers. <https://github.com/luizr/mqtt-malaria> (2019)
 105. Lyon, G.: Nmap: the network mapper. <https://nmap.org/> (1997)
 106. Evilsocket: bettercap: a powerful, modular and portable network tool. <https://github.com/bettercap/bettercap> (2015)
 107. Oliveira, I.: macof: a tool for flooding the network with random mac addresses. <https://github.com/ianluetke/macof> (2003)
 108. Eclipse Mosquitto mosquitto.org. <https://mosquitto.org/>
 109. Breiman, L., Friedman, J., Stone, C.J., Olshen, R.A.: *Classification and Regression Trees*. CRC Press, New York (1984)
 110. Breiman, L.: Random forests. *Mach. Learn.* **45**(1), 5–32 (2001)
 111. Duda, R.O., Hart, P.E.: *Pattern Classification and Scene Analysis*. John Wiley & Sons, New York (1973)
 112. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. *Nature* **323**(6088), 533–536 (1986)
 113. Ptacek, T.H., Newsham, T.N.: Insertion, evasion, and denial of service: eluding network intrusion detection. Technical report, Secure Networks, Inc. (1998). <https://www.icir.org/vern/Ptacek-Newsham-Evasion-98.pdf>
 114. Handley, M., Paxson, V., Kreibich, C.: Network intrusion detection: evasion, traffic normalization, and {End-to-End} protocol semantics. In: 10th USENIX Security Symposium (USENIX Security 01) (2001)
 115. Open Information Security Foundation (OISF): Suricata User Guide: Protocol Anomalies Detection. (2024). <https://suricata.io/documentation/>
 116. Shankar, U., Paxson, V.: Active mapping: Resisting nids evasion without altering traffic. In: *Symposium on Security and Privacy*, pp. 44–61. IEEE (2003)
 117. Alderman, I.: *Network Intrusion Detection Systems and Evasion Techniques*. CS 736 Security Seminar, University of Wisconsin-Madison
 118. Moore, D., Voelker, G.M., Savage, S.: Inferring internet denial-of-service activity. In: 10th USENIX Security Symposium, (2001)
 119. Firdous, S.N.: *Vulnerability analysis and intrusion detection for mqtt-based iot networks*, Edith Cowan University (2021). (PhD thesis)
 120. Mutz, D., Kruegel, C., Robertson, W., Vigna, G., Kemmerer, R.A.: Reverse engineering of network signatures. In: *Proceedings of the AusCERT Asia Pacific Information Technology Security Conference*, (2005)
 121. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Madotto, A., Fung, P.: Survey of hallucination in natural language generation. *ACM Comput. Surv.* **55**(12), 1–38 (2023)
 122. Ahmad, Z., Shahid, A.S., Thurasamy, R.: Network intrusion detection system: a systematic study of machine learning and deep learning approaches. *Trans. Emerg. Telecommun. Technol.* **32**(1), 4150 (2021)
 123. Mahbooba, B., Timusk, M., Kamal, L., Gošević, M.: Explainable artificial intelligence (xai) to enhance trust management in intrusion detection systems using decision tree model. *Complexity* **2021**, 1–11 (2021)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.